PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

AI-BASED TOOL FOR PRELIMINARY DIAGNOSIS OF DERMATOLOGICAL MANIFESTATIONS

A PROJECT REPORT

Submitted by

LAKSHMI PRIYA NARAYAN- 20221CSE0374

HEMANTH K- 20221CSE0046

VANKAM VISWA SAI- 20221CSE0045

Under the guidance of,

MR. JERRIN JOE FRANCIS

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

DECEMBER 2025

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "AI-Based Tool For Preliminary Diagnosis Of Dermatological Manifestations" is a bonafide work of LAKSHMI PRIYA NARAYAN(20221CSE0374), HEMANTH K(20221CSE0046) and VANKAM VISWA SAI(20221CSE0045) who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-26.

Mr. Jerrin Joe Francis
Project Guide
PSCS
Presidency University

Mr. Muthuraju V
Program Project Coordinator
PSCS
Presidency University

Dr. Sampath A K
Dr. Geetha A
School Project Coordinators
PSCS
Presidency University

Dr. Blessed Prince P
Head of the Department
PSCS
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS & PSIS
Presidency University

Examiners

Sl. No.	Name	Signature	Date
1	Aadil Feroz		6-12-25
2	Bikram Saini		05/12/2025

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING at Presidency University, Bengaluru, named Lakshmi Priya Narayan, Hemanth K and Vankam Viswa Sai, hereby declare that the project work titled "AI-Based Tool For Preliminary Diagnosis Of Dermatological Manifestations" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

LAKSHMI PRIYA NARAYAN	USN: 20221CSE0374	Lakshmipriya.N
HEMANTH K	USN: 20221CSE0046	
VANKAM VISWA SAI	USN: 20221CSE0045	

PLACE: BENGALURU

DATE: 5/12/2025

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide Mr. **Jerrin Joe Francis, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Blessed Prince P, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, and Dr. Geetha A, PSCS** Project Coordinators, **Mr. Muthuraju V, Program Project Coordinator**, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

LAKSHMI PRIYA NARAYAN

HEMANTH K

VANKAM VISWA SAI

Abstract

Skin diseases are a universal health issue worldwide where detecting cases early is required yet traditional methods are often slow, subjective and resource constrained. This project is a deep learning-based system that classifies dermatological lesions by analyzing images. The entire mechanism operates using a custom CNN model that has been modified and trained on TensorFlow and Keras until it starts acting. The model performs training and testing until the best accuracy is achieved and saves it. The initial data was highly unbalanced (by a large margin benign over melanoma), hence RandomOverSampler was introduced to replicate the infrequent samples to balance the classes. The dataset used consists of images that are 28x28 grey images. The best model is saved as best_model.h5 including a clean, multi-page Streamlit web app which depicts the model's performance in the best way possible.

The user is provided with an opportunity to upload a picture of a skin lesion in the web app, and the results will appear on the screen consuming least amount of time. It performs accurately with an accuracy of 97.79%, it downsizes the image to 28x28 and normalizes the pixel values directly as it would be during training, and sends it directly to the model. The results appear within a few seconds with the classification (benign or malignant) and a definite level of confidence and a brief and responsible note that users should seek the advice of a dermatologist about anything. Such performance on a task as important as melanoma detection, causes the entire system to feel sound and prepared to be used in the actual world. This is justified with a sophisticated classification report and Macro-average Area Under the Curve (AUC) of 1.00, which is close to perfect value with all the seven disease groups. The model presents the feasibility of a compact, AI-based device that can be used as an affordable, scalable, and reliable auxiliary diagnostic tool, which can contribute immensely to computer-aided diagnosis and teledermatology creation.

Table of Content

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
AKIEC	Actinic Keratoses and Intraepithelial Carcinoma
AUC	Area Under the Curve
BCC	Basal Cell Carcinoma
BKL	Benign Keratosis-like Lesions
BN	Batch Normalization
CNN	Convolutional Neural Network
DF	Dermatofibroma
DL	Deep Learning
EDA	Exploratory Data Analysis
FN	False Negative
FP	False Positive
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
HAM10000	Human Against Machine with 10000 training images
IEEE	Institute of Electrical and Electronics Engineers
IoMT	Internet of Medical Things
ISIC	International Skin Imaging Collaboration
KNN	K-Nearest Neighbors
MEL	Melanoma
ML	Machine Learning
NV	Melanocytic Nevi
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
ROC	Receiver Operating Characteristic

Chapter 1

INTRODUCTION

Early diagnosis of skin-related diseases is very crucial for better treatment and higher survival rates. However, the number of experienced dermatologists and relevant diagnostic equipment is seriously insufficient in many places, especially in rural and underdeveloped parts of the world. Therefore, such a shortage causes delays, wrong treatment, and can even be life-threatening. Moreover, there are cases where skin conditions do have a similar look, making it difficult for even an experienced doctor to diagnose.

The primary aim of this project is to develop a smart and automated system that can precisely classify various skin diseases based on images captured by a dermatoscope. The name of the project is DermaDetect and its primary aim is to construct a robust Convolutional Neural Network (CNN) model capable of analyzing skin lesion images and assigning the most probable disease with a very high degree of accuracy. Besides its efficiency in disease prediction, the project intends to provide a basic web interface via Streamlit, which enables the users to upload images in real-time, receive automatic analysis, and immediately get the diagnostic results that are accessible to the doctors and the public.

1.1 Background

Skin lesion diagnosis is mostly relied on visual treatment, but Artificial Intelligence integration has bought a significant change to this aspect. Recent studies indicate that there is higher chances of Artificial Intelligence and Machine Learning are going to completely automate the process of skin disease detection and improve efficiency [1]. In order to make this innovative tool more reachable, researchers have increased their efficiency to achieve the best possible software to be deployed such as JILDYA_Net who have shown that lightweight architectures are totally capable of achieving higher accuracy [2]. There are some models such as DeepSkinNet that have been modified to be specifically used for skin cancer detection [3]. Such models are being deployed helping in the advancement of telemedicine [4]. Ensembling stack approaches are also being adapted to combine and determine the best of multiple predictions [5]. Ultra-low latency diagnostics are implemented on resource deficit devices [6].

As this domain grows, multiple methodologies have been bought into light where researchers have come up hybrid models which could be a combination of CNN and KNN to increase the overall performance of the model [7]. Few have done improvements to already existing DNN to extract more and more features [8]. Self-attention mechanism allows the models to notice global context [9]. Up-sampling and data augmentation were used to overcome the issue of class imbalance [10].

Multiple deep learning methods were introduces to set a baseline accuracy for the comparison of model's performance [11]. One of the architectures that is EfficientNet-B7 was further optimized for a higher accuracy [12]. The concept of Explainable AI which used tools such as Grad-CAM were used which added a great sense of innovation and convenience to the domain [13]. Standard deep learning frameworks [14] and also multi-class models [15] were proof of advancements in the dermatological field. HAM10000 has been globally used as base ground for testing and improvements [16]. As time goes ahead multiple researchers have come up with mobile friendly software [17] and also DenseNet was part of the evolution [18] followed by InceptionV3 used for melanoma detection [19]. All of these have been a great contribution to train on larger dataset and were successfully implementation of skin-based image classification [20].

1.2 Statistics

Worldwide, skin diseases have been a significant component of health issues, more so in disadvantaged regions that lack sufficiently equipped doctors to handle skin ailments. According to The World Health Organization, at any given time, a staggering total of close to 900 million people globally bear some form of skin diseases, thus ranking these conditions as the most frequent non-fatal health concerns [1]. The 2019 Global Burden of Disease Study ranked diseases of the skin and subcutaneous tissues as the 4th largest cause of non-fatal health problems, thereby emphasizing their societal and economic impact [2].

In the Indian scenario, skin problems have been identified as a leading cause of hospital outpatient visits, accounting for roughly 6-12% of the total visits [3]. The common skin ailments of eczema, acne, fungal infections, and pigmentation have been found by studies to prevail in the hot and humid regions such as India. They are attributable to factors such as climate, pollution, and hygiene [4]. In addition, India accounts for an annual incidence of close to one million new cases of skin cancer, and melanoma, the most lethal form of skin cancer, is

rising rapidly due to lifestyle changes and increased sun exposure, among other reasons [5]. However, the country has an inadequate number of dermatologists where the ratio stands at one per 100,000 people, whereas, in developed nations, the ratio is one per 25,000 people [6]. Due to the shortage of specialists, it becomes difficult to obtain a diagnosis timely and accurately, thus, rural and semi-urban areas that lack well-equipped medical facilities are mostly affected. The growing trend of skin disorders coupled with the shortage of specialists underscores the urgency of having a device capable of diagnosing dermatological conditions in an automatic, dependable, and simple manner.

The model includes the technology which incorporates AI to assess pictures, is a step in the right direction to bridge the gap through enabling early identification as well as screening even in locations that are devoid of dermatologists. Installing such technologies into telemedicine platforms or primary healthcare centers would undeniably have a great positive effect in the diagnosis coverage, patient wait times reduction, and ultimately the saving of lives through early intervention.

1.3 Prior existing technologies

Various automated diagnostic systems have been created around the globe to deal with the skin lesion classification and each one is characterized by the trade-offs between computational complexity and accuracy versus feasibility to deploy the system. One of the dominating categories of solutions is the high-performance deep learning architectures. DeepSkinNet [3] architectures and systems based on EfficientNet-B7 [12] offer full classification capabilities with the help of huge pre-trained feature extractors that can identify subtle dermatological patterns. In the same way, the recent work involving the use of dense Net backbones [18] and InceptionV3 [19] has shown state-of art accuracies of above 90 percent on benchmark data sets. Nevertheless, these so-called heavy models are generally expensive to deploy in terms of computational hardware, like high-end GPUs, and they need specialized technical knowledge to fine-tune. Also, such complicated architectures are sometimes black boxes, i.e. hard to interpret or adapt to a particular, resource-constrained clinical setting, thus limiting their use in telemedicine over a network.

Another type of academic research is hybrid and ensemble systems. Sharma and Reddy used ensemble stacking techniques, which involve multiple models to improve the reliability and decrease variance [5]. On the same note, Ali and Raza came up with a hybrid architecture of the HAM10000 data set through a combination of a Convolutional Neural Network (CNN) to

extract features with a K-Nearest Neighbors (KNN) classifier and obtained strong output [7]. Although these techniques are effective in enhancing the process of generalization, they pose serious complications to architecture. The need to train, execute and sustain various models or different algorithms concurrently adds inference latency and overhead to the system, thus less optimal in real-time, quick-response applications where the diagnostic response is needed. Alternatives that are resource constrained and mobile-friendly put more emphasis on efficiency other than raw power.

Lightweight architectures such as MobileNet based systems [17] and JILDYA-Net [2] use lightweight designs to enable edge device classification. Even state-of-the-art innovations have been able to realize ultra-low latency diagnostics on specialized hardware, including quantization-aware neuromorphic architectures [6]. Nonetheless, one of the common constraints in virtually all categories (heavy, hybrid and lightweight) is the sensitivity to the extreme class imbalance in medical datasets. According to Kumar and Kumar [10], without an exercise in data engineering, even advanced models grow biased to majority classes such as melanocytic nevi. Possibly the reason why model architecture alone-based methods do not tackle this underlying issue of data distribution is that they achieve high aggregate accuracy but poor sensitivity of rare, fatal conditions. The proposed model system fills these gaps as it focuses on the data-centric approach paired with a bespoke lightweight system. Compared to heavy transfer learning models [15], [20], this model makes use of a specially designed Convolutional Neural Network (CNN) that particularly aims at low-resolution (28x28 pixel) input. The design option drastically minimizes the number of parameters to roughly 337,000 allowing its inference in seconds on normal CPUs without compromising diagnostic quality. More importantly, the methodology incorporates an effective Random Oversampling technique to correct the class imbalance during the training phase, such that the model is trained to detect rare malignancies as well as frequent benign lesions. With the choice of this balanced, efficient engine and the user-friendly Streamlit interface, the functionality of which might be likened to the integration that exists in the field of AI-driven IoMT solutions [4], the proposed system can be turned into a practical, scalable second opinion tool that can be deployed in non-specialist healthcare facilities.

1.4 Proposed Approach

Various automated diagnostic systems have been created around the world to deal with lesion classification on the skin with each defining their own tradeoffs regarding computational

complexity, accuracy, and deployment ability. One of the most prevalent types of solutions is high-performance deep learning architectures. Other architectures, like JILDYA-Net [2] and DeepSkinNet [3], offer a broad scope of classification, using new encoders and custom layers to reach state-of-the-art accuracies of over 98% on benchmark data. Equally, much has been accomplished with current backbones such as EfficientNet-B7 [12] and DenseNet [18], which make use of massive in-trained feature extractors to identify minor dermatological patterns. Nonetheless, they often need a lot of computational resources, including high-end GPUs, to be deployed and fine-tuned; they also often require specialized technical skill. Also, such complex architectures tend to be black box, and as such, they are hard to read or tailor to particular, resource-limited clinical settings [13].

Another type of research is hybrid and ensemble systems. Sharma and Reddy used ensemble stacking methods, where many models would be used to reinforce the predictions of each model [5]. In the same manner, Ali and Raza have created a hybrid HAM10000 dataset [16] (Convolutional Neural Network (CNN) to extract features and KNN to classify) that showed correct results [7]. Although the techniques are effective in reducing variance and enhancing generalization, they come with a lot of architectural complexity. Consequently, the need to train, maintain and execute various models or dissimilar algorithms concurrently raises inference latency and they are not the most optimal in real-time, quick-response in distant telemedicine setting. Mobile-friendly and resource-constrained systems, e.g. MobileNet-based systems [17] and quantization-aware neuromorphic architectures [6], are resource-optimized in terms of efficiency rather than raw power. These platforms make it possible to classify edge devices with the lowest latency. Nevertheless, a common weakness of most of the categories, namely, heavy, hybrid, and lightweight, is that they are sensitive to the extreme class imbalance of medical datasets.

According to Kumar and Kumar [10], unless the data engineering is done rigorously, even the sophisticated models will become biased to majority classes such as the melanocytic nevi. Conventional models which make use of only model architecture tend to ignore this underlying data distribution issue causing high total accuracy but low sensitivity to rare and fatal conditions. The proposed model system fills these gaps, as it focuses on a data-centric approach to architecture instead of focusing on the complexity of the architecture itself. In contrast to heavy transfer learning models [19], [20], is based on a custom-made, low-resolution (28x28 pixel) Convolutional Neural Network (CNN) that learns on low-resolution images. The design option drastically decreases the number of parameters and allows quick inference on

conventional CPUs without compromising diagnostic accuracy. More importantly, the method incorporates a powerful Random Oversampling policy to correct the lack of classes before training so that the model learns to detect rare malignancies just as well as common benign lesions. With a balanced and efficient engine, and a friendly Streamlit interface, the proposed system turns into a non-hypothetical prototype and becomes an available and scalable second opinion aid that may be implemented in non-specialist clinical contexts.

1.5 Objectives

Specific, measurable, achievable, relevant and time-bound objectives are utilized to develop the Automated Skin Disease Classification System in its objectives that will meet technical, operational and organizational needs.

Objective 1: Build Performing Data Engineering and Preprocessing Pipelines.

Architect and develop high-fidelity data processing pipeline that would be able to standardize various dermoscopic inputs into a single format, which would be analyzed by deep learning. Converts raw image data into 28x28 pixel RGB tensors that are z-score standardized (mean=0, std=1) in order to achieve consistent model convergence. The key issue to consider is the problem of class imbalance since it is important to adopt a strict Random Oversampling approach and increase the size of the training sample by 46,935 to provide equal opportunities to all seven diagnostic categories. Support an efficient, end to end, preprocessing pipeline that manages the ingestion, resizing, normalization and reshaping of data with minimum latency at both test time and training times.

Objective 2: Increase Multi-Class Diagnosis with High-Accuracy Deep Learning.

Design and train a specialized lightweight Convolutional Neural Network (CNN) that can identify the skin lesions in seven different categories with a minimum accuracy of 97 on unknown test samples. Adopt a hierarchical architecture of feature extraction with sequential blocks of convolutional layers, Max Pooling and Batch Normalization to learn complex dermatological images, including lesion boundaries and texture gradients. Add powerful regularization strategies, such as Dropout (Dropout rates 0.25 and 0.50) to avoid overfitting and guarantee generalization. Check model performance with extensive metrics, reaching a Macro-Average Area Under the Curve (AUC) value of around 1.00 and high sensitivity of the model to both rare and common disease groups.

Objective 3: Deliver Intelligible User Interfaces and Clinical Decision Support.

Develop and deploy an expert web-based application built on Streamlit and offer a user-friendly interface to make real-time diagnostic analysis. Create interactive capabilities to enable users to upload high-resolution dermoscopic images and get the results of classification with the confidence probability scores in real time. Include an intelligent guidance system that converts model predictions into clinical actionable advice (e.g. Urgent biopsy advised in Melanoma), and color-coded notifications to prioritize high-risk cases. Make the interface interactive and user-friendly, with no technical support and skills necessary to use it among non-specialist healthcare providers.

1.6 SDGs

The proposed project is an AI-based skin disease detection system. It connects with several of the United Nations Sustainable Development Goals (UN SDGs) that focus on global health, innovation, and fairness. Specifically, this project supports Goal 3: Good Health and Well-being and Goal 10: Reduced Inequalities (United Nations, 2015).

SDG 3: Good Health and Well-Being

Goal 3 of the Sustainable Development Goals is all about giving everyone a fair shot at staying healthy, no matter their age or where they live. Skin cancer and especially melanoma has quietly climbed the ranks to become one of the most common cancers pretty much everywhere. The frightening aspect does not lie in the cancer itself but in the fact that the better the likelihood is to beat the illness the sooner you discover the existence of such a disease. Find it early enough and the survival rates skyrocket. That is where something as DermaDetect does come in. It places a trustworthy AI control in the pocket of any person: snap a picture, receive an answer within a few seconds, and realize whether you are going to a clinic immediately or you are free to breathe and rest now. It is not about eliminating physicians, it is about ensuring that more people access a doctor before it is too late to correct the situation. Eventually, that would be directly in line with Target 3.4: reducing untimely deaths due to such diseases as cancer by detecting them earlier and bringing people to appropriate care sooner. One less family losing someone way too early is a small miracle that will allow even a single person in a distant village to be treated in time. That's the whole point.

SDG 10: Reduced Inequalities

Goal 10 is simply ensuring that the world does not continue to become divided into the haves and the have-nots particularly when it comes to basic necessities like survival and good health. Today, in a large metropolis having a good insurance, it is not that big of a deal to notice a suspicious mole at an early age: you find a dermatologist, have him examine it, fine. But if you're in a rural area, hours from the nearest specialist, or in a country where there just aren't enough skin doctors to go around, that same mole can turn into a death sentence simply because nobody saw it in time.

This tool doesn't fix the shortage of doctors, but it does something almost as important: it puts a reasonably trustworthy first opinion in everyone's browser, for free, on whatever phone or old computer they happen to have. A farmer in a village with spotty internet can still upload a photo, get a clear "this looks worrying, go see someone" or "probably fine, but keep an eye on it," and make a much smarter decision about whether to spend a day and a bunch of money traveling to the city. That's real-world inequality shrinking, one quick skin check at a time. When people who've always been on the wrong side of the healthcare gap suddenly get the same early warning the city folks take for granted, that's Goal 10 actually happening between urban and rural populations. United Nations' plan for a more sustainable future.

Fig 1.1 Sustainable development goals

1.7 Overview of project report

The report gives a detailed insight into the designing, designing, and development of the model, an AI-based solution to skin disease detection.

Chapter 1 gets the ball rolling by providing the scene: why would anyone care about skin cancer in the first place, what the issues of the real world would look like (particularly in the regions where a visit to the dermatologist is like winning the lottery), and what is it that this project seeks to repair. It also connects it to some of the UN Sustainable Development Goals in order to make it apparent that it is not just another student project that was conducted by a student who wants to see the needles move on the real-world issues.

Chapter 2 explores extensively what is already there. It traces through the history--the early days of people manually building features and feeding them to SVMs and Random Forests, up to the great CNN revolution of that infamous 2017 Stanford paper, and more recently the lightweight models and Vision Transformers. It is simply a journey of all that was tried, what worked reasonably and where things fail (unbalanced datasets, not so effective in cases of darker skin, black-box predictions that doctors do not put trust in, etc.).

Chapter 3 gets technical: what datasets was used, how the images were cleaned and augmented, why EfficientNet was chosen over the classic ResNet suspects, how the training was configured (transfer learning, balanced sampling, all the other tricks), and what measures really mattered towards determining whether the model was ready to see real skin photographs.

Chapter 4 is on transforming the trained model into a reality where people can even click on it. It includes the details on how the entire package was stitched into a Streamlit web app that can show pictures on a shaky 3G connection and work, how the uploads are processed behind the scenes, and how it actually goes to be the difference between one user drops picture and gets an answer with a confidence score along with a friendly reminder about seeing a doctor in case it looks bad.

Chapter 5 presents the visualized view of different approaches, structure and flow of the project ensuring smooth and clear architectural visual representation.

Chapter 6 shows what real tools were employed, how it all cost and how all this was balanced with classes and life did not go haywire. It is concluded in Chapter 7: here is what we successfully managed to peel away, here is why it is important, and here is what might follow it: a proper offline mobile application, adding more diseases than just melanoma.

Chapter 7 includes the evaluation metrics that are used to determine the working ability of the model, what influences it to perform better or bad and which approach gave what accuracy on what type of data.

Chapter 8 discusses the social, legal, ethical, sustainability and safety parameters of the model that depicts what influence it has in each aspect. How it contributes towards positive betterment of the world in a transparent and clear setting.

Chapter 9 concludes the entire journey of building an efficient model to obtain best results and achieving a state of art model that solves a striving global issue to the best of its ability.

Chapter 2

Literature Review

Deep Learning-based automated system development has advanced fast to classify skin disease. The chapter gives an in-depth review of the current researches synthesizing the literature of 20 peer reviewed articles to establish gaps that the proposed system will fill.

2.1 General AI in Oncology

Aftab et al. [1] also have given a general overview of AI use in oncology, and the shift of the conventional diagnostic tools to deep learning networks such as VGG and ResNet. Their research highlighted the fact that despite the fact that these models can provide better diagnostic accuracy, there are still major issues to deal with such as data bias and the black box nature of AI that does not support clinical interpretability and trust.

2.2 New Lightweight Architectures

Laouarem et al. [2] proposed a lightweight architecture JILDYA-Net which has been developed with the aim of efficiency and this achieves accuracy of 98.80 percent on the HAM10000 dataset. Although very effective, the authors have observed that the model was sensitive to class imbalance where the performance was poor on underrepresented disease categories. In reference [3], Mehta et al. suggested an ad-hoc deep learning model, DeepSkinNet, that is specific to cancer of the skin. Their architecture had excellent feature extraction performance but depended heavily on particular preprocessing stages that could narrow its generalization to different clinical settings.

2.3 Telemedicine and Ensemble Systems

Mohsin et al. [4] created a remote triage IoMT solution based on an AI that combines sensory and non-sensory data to reach 98.4% accuracy. Nevertheless, the fact that the system is based on simulated environments to be validated also raises some doubts about its ability to be effective in a chaotic real-world clinical environment. Sharma and Reddy [5] examined techniques of ensemble stacking, or multi-deep-learning models, which improve the classification performance through the combination of multiple deep-learning models. Although useful in reducing variance, it was found to add big computational overhead and could not be used in real-time operation on devices with resource constraints.

2.4 Special Hardware and Hybrid Models

Wang et al. [6] proposed a quantization-conceptual neuromorphic architecture with ultra-low latency (1.5 ms). Although it is novel in terms of efficiency, the necessity to install specialized neuromorphic hardware in the present condition restricts its use in the general medical care facility. Ali and Raza [7] suggested a hybrid system, which involves CNNs to extract the features and K-Nearest Neighbours (KNN) to classify the features. This was a good accuracy method but carried a two-step inference process which made it more latent than end-to-end deep learning models.

2.5 Advanced Neural Mechanisms

Gupta and Singh [8] examined the ways to improve conventional CNNs by hyperparameter optimization. Their effort provided a better accuracy but with diminishing returns of tuning without dealing with the underlying data quality problems. Khosro Rezaee and Hossein Ghayoumi Zadeh [9] integrated Transformers with CNNs to utilize the attention mechanisms to themselves. In spite of attaining 97.48 percent accuracy, the Transformer component is not particularly mobile friendly due to its high computational cost.

2.6 Data Engineering Strategies

The problem of class imbalance seen in literature as completely negative was also considered critical in a study by Kumar and Kumar [10], who showed that up-sampling, as well as augmentation, is a necessity to achieve reliable performance. Their results validated that in the absence of such interventions, the models are highly biased in favor of majority classes a phenomenon which is usually ignored in studies of pure architecture. Patel et al. [11] gave a literature review of these methods, clearly showing the requirement of data-centric methods, as compared to the purely model-centric methods.

2.7 Transfer Learning at a high level of performance

Prakash et al. [12] have used EfficientNet-B7 model in order to obtain good diagnostic precision. Nevertheless, the large parameter size of B7 (more than 64 million) renders it computationally exorbitant to edge devices. The authors Saiful Islam Badhon et al. [13] emphasized the Explainable AI with the help of Grad-CAM to detect the lesion outlines. Though it can help in increasing interpretability, the use of post-hoc visualisations does not necessarily make the underlying model more accurate as regards to classification. Sahu and

Jain [14] compared mainstream deep learning models to detect melanoma, which provides concrete baselines but not much innovation in terms of architectural design.

2.8 Benchmarking on HAM10000

Das et al. [15] used EfficientNets to a multi-class classification task and reported that it was hard to discriminate between non-melanocytic visual similar lesions. Rao et al. [16] had done a lot of experiments on the HAM10000 data set using standard backbones. Their findings emphasized the fact that the accuracy was quite good although the sensitivity of rare classes such as the Dermatofibroma could not be optimum unless a special balancing approach is adopted. Mobile and Dense Architectures

2.9 DenseNet Integration

Zhang and Park [17] explored MobileNet to classify skin lesions, and it was found that speed is a priority in mobile applications. The trade-off was a significant loss in accuracy relative to larger models, but was efficient. Nair and Raj [18] compared the DenseNet architectures, which enhanced feature reuse by dense connections. Nevertheless, it was found that the high memory cost of DenseNet in training is a challenge to researchers with limited hardware.

2.10 Foundational Studies

Another breakthrough study by Tan and Le [19] revealed the effectiveness of InceptionV3 in detecting melanoma, the introduction of transfer learning as one of the working strategies. Nonetheless, they were more binary (malignant vs. benign), which would not be useful in multi-class clinical environment. Lastly, a validation of pre-trained models of skin lesion was offered at an early stage by Das and Dutta [20]. Their efforts were based on older architecture that have since been overtaken in efficiency and accuracy but it highlighted the challenge that persisted of adapting domains between natural and medical images.

Summary of Literatures Reviewed

Table 2.1 Summary of Literature Reviews

Title of the Paper	Name and Author	Method Used	Accuracy / Performance	Merits	Demerits
Skin Disease Classification using Deep Learning and Ensemble Stacking Approach	S. Sharma and P. R. Reddy (2025)	Ensemble Stacking (CNN + others)	98.0%	High accuracy through hybrid model; improved feature diversity	Computationally heavy; limited generalization to unseen diseases
DeepSkinNet: A Deep Learning Model for Skin Cancer Detection	A. Mehta, N. Kaur, and R. Tiwari (2025)	DeepSkinNet CNN	97.35%	Robust architecture; reduced overfitting; strong on small datasets	Requires large GPU resources; limited dataset variety
Enhanced Skin Cancer Classification Using Deep CNN	M. Gupta and L. Singh (2024)	Custom CNN	96%+	Simple yet effective CNN; efficient feature learning	Lower precision on rare lesion types
Skin Lesion Classification Using Deep Learning Techniques	K. Patel, R. Bhatt, and S. Verma (2024)	CNN with class balancing	96%+	Balanced dataset improves classification fairness	Training time increases with augmentation
Employing Up-Sampling and Augmentation for Imbalanced Skin Lesion Classification	R. Kumar and V. Kumar (2024)	Transfer Learning (ResNet-50)	~95%	Handles imbalance effectively; improved recall	Slight loss in overall accuracy due to over-augmentation

Improved Skin Cancer Detection Using EfficientNet-B7	S. Prakash, D. Banerjee, and A. Saha (2024)	EfficientNet-B7	95%+	High efficiency and depth scaling; robust to noise	Requires fine-tuning; sensitive to hyperparameters
HAM10000 Skin Disease Detection and Classification Using Hybrid CNN and KNN	A. Ali and M. Riaz (2024)	CNN + KNN Hybrid	~94%	Combines spatial and statistical learning	Limited scalability; complex training pipeline
Skin Lesion Detection and Classification Using Machine Learning	J. Chen, L. Wang, and Y. Liu (2023)	SVM, Decision Tree, CNN	94.8% (SVM)	Comparative model evaluation; interpretable results	Classical ML models underperform on large datasets
Skin Cancer Detection by Using Deep Learning Approach	P. Sahu and R. Jain (2023)	Custom CNN	78%	Simpler architecture; quick training	Low accuracy; limited feature extraction
Multiclass Skin Cancer Classification Using EfficientNets	B. Das, T. Rahman, and S. Chatterjee (2022)	EfficientNet B0–B7	93–94%	Scalable across multiple architectures; strong baseline	Limited preprocessing; minor class confusion

Chapter 3

METHODOLOGY

In this chapter we will look into the methodology deployed to build the AI based tool for preliminary diagnosis of dermatological manifestation which works on the base of deep learning, machine learning and Artificial Intelligence. The model was deployed using different phases. The methodology explains a details step by step process to implement a proper and efficient model, which is capable of performing best in its field.

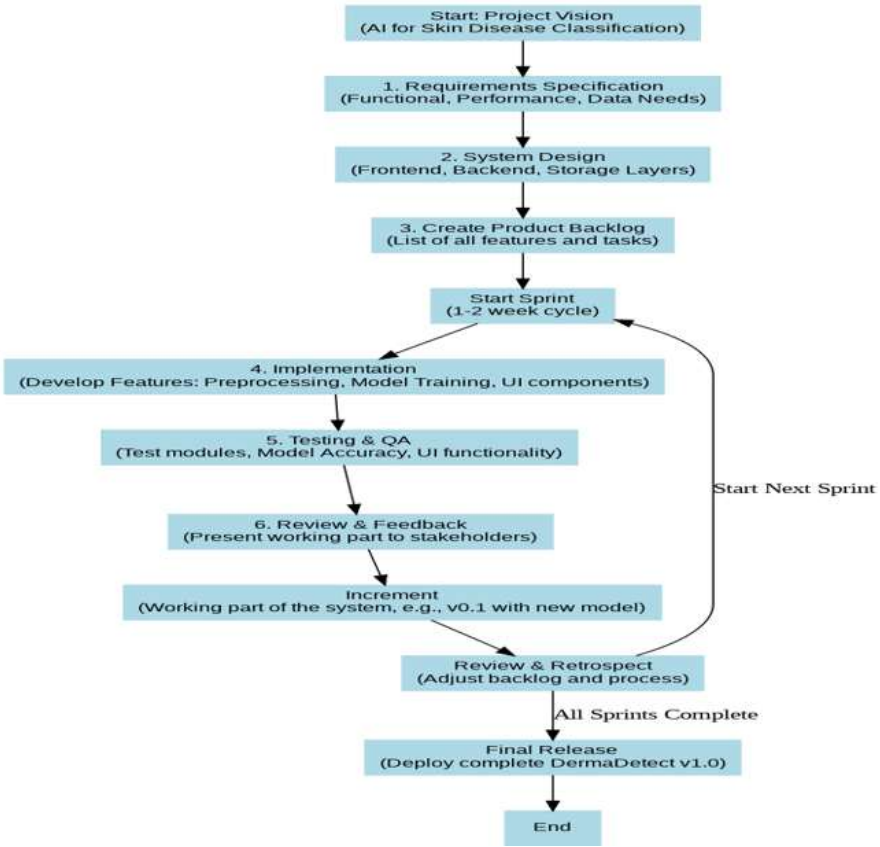


Fig 3.1 Agile Development Methodology

3.1 Requirements Specification

The first step was devoted to the determination of the functional, performance, and data requirements to develop a powerful diagnostic tool.

3.1.1 Hardware requirements:

To achieve efficient model training and fast inference, the development was carried out on a local machine with a high level of performance with the following specifications: Processor:

AMD Ryzen 7 7435HS (to be fast in processing data and multitasking). GPU: Nvidia GeForce RTX 3050 (used to help with Deep Learning calculations). RAM: 16GB (to store the oversampled dataset in memory).

3.1.2 Software Requirements:

Google Colab:

This is where the web interface was implemented and the initial testing of the interface was done. Kaggle Notebook: utilized to implement the backend in which you can access dataset repositories and cloud GPUs to execute heavy training. Scikit-learn: This is necessary to do data engineering, in particular to use RandomOverSampler to correct class imbalance. Streamlit: The main structure of building the interactive dashboard, where it is possible to upload images and get real-time predictions.

3.1.3 Data Requirements:

The project has found a critical data issue: the high level of class imbalance in HAM10000 dataset. The type of Melanocytic nevi was immensely dominant in comparison with rare and deadly illnesses such as Melanoma. An extreme demand was placed to solve this bias through a data-centric method (Oversampling) and not just algorithmic adjustments. This makes the model not prejudiced against the majority group and is able to produce dependable results in detecting rare yet life-threatening lesions in practice.

3.2 System Design

At this stage, the application architecture was to be designed as a clean, three tier modular system. This is a component-based solution, which is scalable and maintenance friendly.

Frontend Layer (Presentation):

The presentation was designed with the help of Streamlit. The decision prevented the intricacy of frameworks such as React, yet provided a multi-page design with a professional look. The important features were a home page provided to help with the context and a Diagnosis Tool to make the main functionality. The interface has the image widget which can be dragged and dropped, the instant result panels, and the confidence score graphs. The layout is responsive, and therefore the tool can be used on desktop and mobile browsers.

Backend Layer (Logic & Inference):

Python, TensorFlow, and Keras are what drive the logic layer. It manages the main functions: input of an image, loading of the trained model in memory and running the preprocessing stream (resizing to 28x28, normalization). The architecture defines a hand-crafted and custom CNN with exactly 337,671 parameters. This design was selected due to this particular input size of 28x28 pixels, as the computation cost of such large pre-trained models as EfficientNet or ResNet is not needed. This design option makes the model lightweight and high-speed.

3.3 Implementation

The phase included the actual language and coded development of the functional modules that were specified in the design phase. Data Acquisition and Balancing: The process starts with the loading of the hmnist2828RGB.csv file that has 10,015 flattened images (2,352 pixels each) loaded. To combat the imbalance in the classes that could be considered inhumane, the RandomOverSampler algorithm was used that will recreate the samples of the minority classes until the distribution is perfectly balanced. Training Pipeline Model: This pipeline involves data rearrangement into the 3D tensors, the use of Z-score normalization, and the custom CNN training. The complete workflow process of loading, resampling, shuffling, and saving of the balanced dataset has been streamlined to be optimally run. It is the final balanced data that allows the model to have high performance on all the classes, and not just the guess of the most common.

3.4 Testing and Quality Assurance (QA)

In this stage, the trained model (Increment) was strictly tested with respect to requirements. Validation: The model was tested on a held-out data (20% of data).

Metrics: The key performance indicators were verified and found to be 97.79% accurate and a Macro-Average AUC of 1.00.

Stability Testing: Training curves were tested to make sure that no overfitting happened (validation loss tracking training loss).

3.5 Deployment & Release

The last step was to roll out the validated system to system end-users. Integration: The model (bestmodel.h5) was trained and deployed with Streamlit frontend.

Chapter 4

PROJECT MANAGEMENT

4.1 Project timeline

The project is about creating a skin-related tool using a type of artificial intelligence called CNN. This AI was trained using a large set of images called HAM10000. The way the project was organized made sure that each step, from getting the data ready to testing the final model, was done properly and in order.

Table 4.1 Project Timeline as per phases

Phases	Timeline	Task Completed
Month 1	July 2025	The initial step was to define the project scope, carry out a thorough literature review of the information about CNNs applied to classifying skin lesions, and obtain the HAM10000 Dataset the legal ownership of which. The eye-opener was the actual Exploratory Data Analysis (EDA). The visualization of the class distribution in the form of plotting revealed painfully clearly how uneven the dataset was: there were classes with thousands of samples and the ones with just a few samples. That one set of bar charts essentially shouted "don't fix me until I fix you," so the next thing that would occur was childhood-in-aggressiveness data-balancing approaches had to occur before any substantial training of any kind could get underway.
Month 2	August 2025	The entire section was concerned with preparing the images in a way they were actually going to be trained. It involved remaking all to the correct (28, 28, 3) format, performing standardization in such a way that pixel values were compatible with the network, and most importantly, getting RandomOverSampler running to make all seven disease classes split perfectly during training and testing. No shortcuts, and equal, untainted data that eventually provided each of the classes with an equal opportunity.
Month 3	September 2025	This step fixed the real network backbone. To ensure the training process was stabilized, we piled up the convolutional blocks topped with

		BatchNormalization and applied the Dropout layers so that we could reduce overfitting even before it began. In addition, the traditional safety nets were introduced: ModelCheckpoint to pick the best weights automatically, EarlyStopping to ensure that we did not spend hours on a plateau, and ReduceLROnPlateau to provide the optimizer with a light push when things slowed down. Those callbacks essentially made a fine architecture one that ran at maximum performance without the need to babysit.
Month 4	October 2025	This involved practical training. It was combined with sparse categorical cross entropy as the loss and Adam to perform the intended task of optimization. We then allowed it to run up to 50 epochs, with the callbacks monitoring all the actions of the process: each step, EarlyStopping when stalling, reducing the learning rate when appropriate, and automatically saving the absolute best weights to bestmodel.h5 each time the validation scores improved. And by the end, it wasn't the possibility of a decent model that we held onto hope, but the one and only best version that was automatically stored and was going to get deployed.
Month 5	November 2025	This was the last step of the process, and the main goal was to test the bestmodel.h5 and to find out what it was capable of. We ran a full evaluation on the held out test set and extracted all the metrics that actually matter, namely: overall test accuracy, a detailed classification report with precision, recall and F1-score of each of the seven classes, a confusion matrix to identify where exactly any mistakes were occurring, and proper multi-class ROC curves with AUC scores in order to demonstrate that the model was not only strong on the common classes but solid across the board. If it were not those deliverables that provided us with the solid evidence that the system was ready to be deployed on real users.
Month 6	December 2025	This level involves writing up and preparing the last viva that would confirm our knowledge in the subject.

The one we chose fits the AI and machine-learning projects nearly perfectly because it is merely a logical progression of any well-constructed data-science project: You begin with data collection and EDA (you cannot build anything before you have peaked the monster in the eye), then proceed directly to preprocessing and balancing (the not-so-glamorous but make-or-buys

part), design and model training, honest evaluation of the model on unseen data, wrap the service into an interface, and finally deploy and plan further. Leaping or mixing up those steps only forms havoc afterwards--believe me, I have done so--so having them in this particular order made everything go forward in an orderly manner without retracking back and ugly surprises. First Phase (Planning and Data): This initial phase consisted of an amassing of the dataset, scraping of the papers surrounding the issue, and this initial scrupulous glance at the figures. The ideal thing would have been to spot the enormous class imbalance at the time, before writing a single line of model code. Early detection ensured that all decisions made after (choice of strategy to follow, metrics to monitor, even the eventual size of the model) were geared towards correcting the underlying issue rather than covering it up in the future. Second Phase (Model and Training): This was the in between that was really going to happen, and the sequence was of the essence.

Then there was the gruesome data-prep: cleaning, reshaping, standardization, and banging the class imbalance into fairness. It was only then that we locked in and created the CNN architecture (number of layers, dropouts, locations of batch-norm). However training was the last and the last resort, since attempting to train on dirty or unbalanced data would have been a complete waste of days. The fact that these steps were kept in order (prep - design - train - tweak) ensured that all pieces truly were built on sound rather than blindly guessing. Third Phase (Evaluation & Documentation): It was the last phase that was concerned with demonstrating that the system had been put into operation and that nothing can be left to doubt. To do this we took the model through a complete battery of tests - overall accuracy, per-class precision/recall/F1, confusion matrix, multi-class ROC/AUC curves etc. - to understand where the model was doing well, and where it still needed to be improved. Then all was duly chronicled: what we did, why we did it, how many we got and what we learned.

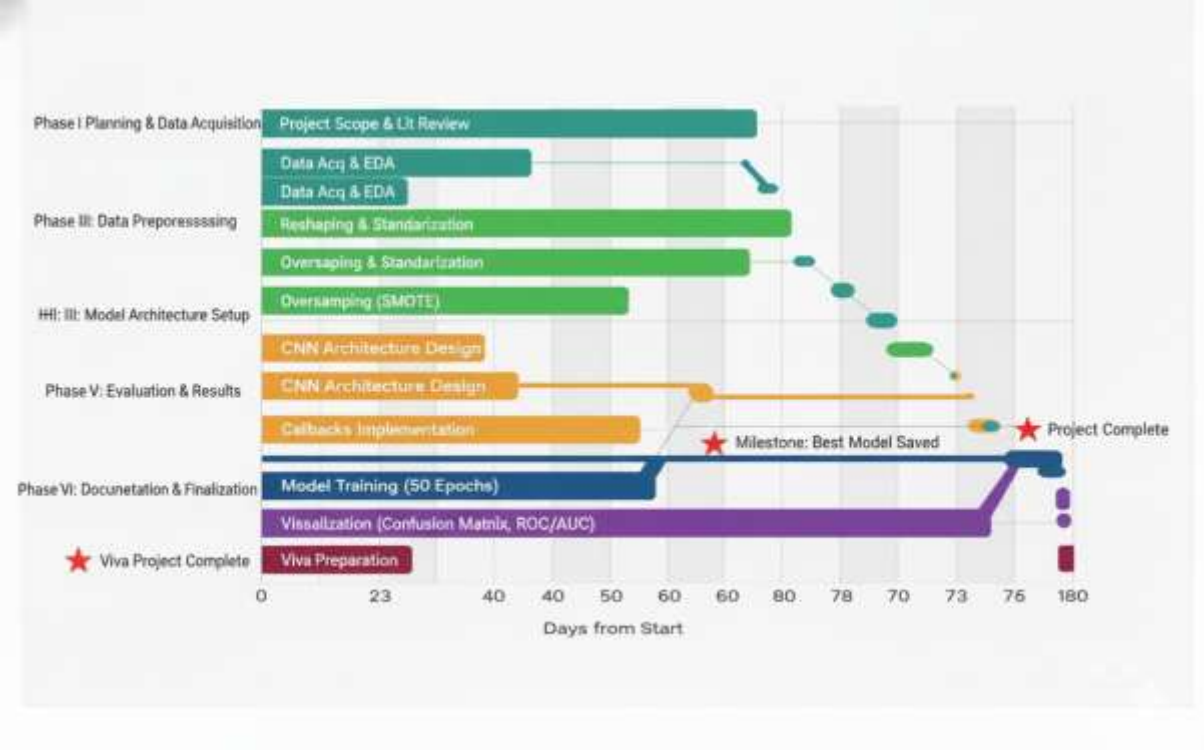


Fig 4.1 Gnatt Chart Visualizing the Project Timeline

4.2 Risk Analysis

We conducted a brief PESTLE analysis to check all the major external factors which might either benefit or disadvantage upon coming out of the real world. Early overview at the political, economic, social, technological, legal, and environmental fronts will allow us to identify risks (such as changing regulations on telemedicine or patchy internet access in clinics in rural areas) and opportunities (increased smartphone adoption, new funding on digital health technologies) before they became unexpected. It simply kept us on track of the project and ensured that we were not making this great thing that no one could actually utilize.

4.2.1 Political Factors

The political aspect includes matters such as government regulations of telemedicine, laws of health-data privacy, or even the state of the country as a whole. The mere appearance of an app such as this model can be either made or ruined by things such as unexpected crackdowns on data sharing of patients, alterations of internet regulation, or budget cuts of digital health projects. With these early of which we should design about the red tape, and these to ensure that even when the political mood changes the tool will continue to work. Risk: Changing Healthcare Regulations Governments continue to change the regulations on data about patients (HIPAA-type laws, GDPR on health data, or state equivalents) and the ways AI can be applied

to anything that even bears a semblance of medical diagnosis. A single change can push weeks of reworking or even shut down the deployment. Mitigation: Since the very first day we constructed the system we made it touch as little personal information as we could- images are processed and deleted immediately afterwards, no accounts, no storage unless the user requests it. All anonymization occurs upon landing. We also put a big sticker on it everywhere: this is a screening device, but not a diagnosis--see a doctor. Such language keeps us within the precincts of the majority of existing regulations until the next revision of the rule.

4.2.2 Economic Factors

Things such as the general economy, interest rates, finances at hand and simple old expenses. Opportunity: Low Cost of Development. Almost all our hardware was free or nearly free Python, TensorFlow, Keras, the public HAM10000 dataset, and the free GPUs of Google Colab when we needed something faster and the cheapest AWS tiers when that was not sufficient. The entire prototype was less expensive than a decent laptop, and since it is all open-source, it will not be like a giant hit when the time to go large comes. Risk Future Commercialization Costs. In case the model becomes a real medical device, the bill will be modified speedily - certified cloud providers, appropriate clinical trials, high-quality datasets, regulatory documents. It is not hard to make those extras jump overnight between student-budget and start-up budget.

4.2.3 Social Factors

In essence the way people think, cultural changes and whether people feel safe leaving a computer to give their opinions on their health or not. Risk: Trust and Acceptance. Many users are still scared when an algorithm informs them that this spot resembles melanoma. In the case the model generates an unsightly false positive (or worse, fails to detect something) the backlash on social media or in the news can be fatal to trust before the tool even gets a fair hearing. The black-box feel and the atmosphere do not help - people would like to know how the machine could say this and that. Mitigation: The tool should be interpretable and understandable. Display explicit performance measures such as the confusion matrix, per-class recall, and ROC/AUC curves to indicate the users precisely what to expect and where the boundaries lie. Include some plain-language explanation and visual heat maps where feasible to allow individuals to understand why the model inclined in one way or the other. Most of all, just repeat it over and over that DermaDetect is merely an assisting tool that points to things

that a real dermatologist needs to inspect, but not the end-all. Turning to that frankness is one step in the right direction as far as trust rather than mistrust is concerned.

4.2.4 Technological Factors

All these have to do with what is available in terms of tech, its readiness, and the ability of the infrastructure to support it. Risk: Dataset Limitations. Using HAM10000 alone would imply processing very small 28x28 size images which appear rather like coloured blobs than actual skin images. At that resolution, a lot of fine details simply disappear, and hence the model levels off very rapidly regardless of how smart the code is. Mitigation: We pushed tricks such as BatchNormalization, Dropout and aggressive RandomOverSampler to their limits to make the most out of what we had. Meanwhile, all the demos and descriptions are transparent: the jumps in accuracy will only be real in a world where we use larger and sharper data sets in the future. Opportunity: TensorFlow, Keras, and the entire Python ecosystem are unbreakable and virtually free. Within an afternoon, you can spin up a working CNN, compared to weeks, and the project proceeded at a rapid pace without struggling to do so. 2.4.5 Legal Factors This includes patents, data privacy legislation and who is held responsible in case of bad things happening. Risk: Liability on Misdiagnosis. Should the app ever issue a statement that it looks fine when it is not or gives someone a false positive that causes them to run to an unwarranted biopsy, the legal headache may be colossal. Mitigation: Disclaimers everywhere: Big, obtrusive disclaimers all over the place: "This is not and should not be taken as professional medical advice. We process images in real time and no image is being saved, we use the HAM10000 license, and the code is clean and open so they will not ask any ownership questions in the future.

4.2.6 Environmental Factors

In essence the extent to which the project is damaging the planet during its operation. Risk: Computational Carbon Footprint. Neural networks can consume power like a wildfire during training, particularly when you authorize hundreds of epochs of the networks running on them. Mitigation: ReduceLROnPlateau and EarlyStopping interrupt training when GPU gains begin to flatten and may save days of trained time. Any further larger implementation we do will be choosing cloud providers that are powered by renewables or carbon-offset credits to allow the footprint to remain as minimal as feasible.

4.3 Team Roles and Responsibilities

The team consists of three members, Lakshmi Priya Narayan, Hemanth K and Vankam Viswa Sai and the workload was equally split and implemented among the team members.

- **Lakshmi Priya Narayan(20221CSE0374):**
  - **Role:** Backend development and testing.
  - **Responsibilities:** Developed a custom CNN architecture adaptable to the dataset performing data pre-processing, training and testing.
- **Hemanth K(20221CSE0046):**
  - **Role:** Performance evaluation and debugging.
  - **Responsibilities:** Verified the working ability of the project and revaluation of the obtained parameters ensuring we have achieved best results.
- **Vankam Viswa Sai(20221CSE0045):**
  - **Role:** Frontend development.
  - **Responsibilities:** Designed an interactive dashboard using Streamlit which enables users to upload their images to obtain results.

4.4 Project budget

Step 1: Tasks and Resource Requirements

Table 4.2 Tasks and Resource Requirement

Category	Required Resource	Justification
Data Source	HAM10000 Dataset	Required for training and validation.
Software	Python, TensorFlow/Keras, Pandas, Scikit-learn, etc.	Core programming and deep learning frameworks.
Hardware/Compute	GPU-enabled processor (Cloud or local lab)	Necessary for training the CNN model efficiently.
Documentation	Standard Office/PDF software	Required for report generation and submission.

Overhead	Internet access, Printing	Basic administrative and communication needs.

Step 2: Team Availability All this was dependent on the time of the day that the team and supervisors were free. This project was an academic one, so the time spent on all the coding, training, and writing were expenses of regular coursework and research time - effectively, sunk costs, no additional funds were required. We also maintained a record of who was allowed to do what at the normal bi-weekly sprint catch-ups.

Step 3: Task Duration To determine the time of things we simply traced through the schedule outlined earlier (Section 4.1): Cleaning the information and initial EDA rounds - approximately 6 weeks. Assembling the model, training it and fiddling with hyperparameters - another 6 weeks A thorough test and a complete write-up - about 8 weeks Any minor costs were spread over the weeks, and it was mostly to cover small working items.

Step 4: Experience and Data We did a couple of ML projects previously, and we already understood that the big money-sinkers are to be avoided: No license costs - all of it is based on free open-source software (Python, TensorFlow, Streamlit, etc.). There are no charges on data - HAM10000 is open-source. Hardly budgeted - university laboratory computers and the free versions of Colab and Kaggle did nearly everything.

Step 5: Set the Project Budget The bottom line at the end of day was pocket change - just a few dollars to do some printing, perhaps a cup or two of coffee, and a couple of small cloud top ups in case Colab got out of control. The heavy lifting was all taken care of by open-source tools and university resources and so we never needed serious funding.

Table 4.3 Project Budget Table

Item	Description	Estimated Cost (₹)	Justification
Software Licensing	Python, TensorFlow, Keras, etc.	₹0	Open-source software.
Data Acquisition	HAM10000 Dataset	₹0	Publicly available dataset.
Hardware/Compute	Use of existing institutional resources.	₹0	Utilized university labs/free cloud platforms.
Administrative Overhead	Printing, Stationery, Final Submission Fees (if any).	₹1,500	Covering physical report and final submission requirements.
Total Estimated Budget		₹1,500 (Minimal)	Highly cost-effective model development.

Step 6: Monitoring of the Project Budget and Team Evaluation. Budget Tracking: Since there is virtually no money on the table, it was very easy to monitor the expenditure. We had to triple-check every week whether no one had inadvertently used the paid API or premium cloud tier, and we had the university laboratories and free Colab credits when we needed them. That was about all--there was never a surprise bill, as such. Team Assessment: We evaluated individual performance by looking at three easy parameters, namely did we meet the time frame we plotted in Section 4.1, did the challenging parts (such as making RandomOverSampler finally balance the classes and the custom CNN code to stick) were accomplished without any drama and most importantly did we achieve the numbers that were important, solid test accuracy and high ROC/AUC scores across all the seven classes. When those boxes were checked, we were certain that the team had been delivered.

Chapter 5

ANALYSIS AND DESIGN

Analysis and design really are two of the most crucial steps when putting any system together. During the analysis, we figure out what the AI diagnostic tool needs to do by looking at the problem and setting clear goals. In the design phase, we plan how to build the system, explaining the structure, parts, and steps that will help reach those goals.

5.1 Requirements

The requirements analysis aimed to gather the essential purpose, behaviour, and limitations needed to create a dependable and working initial diagnostic tool. Since the project is centered around using deep learning with an already available dataset, the main requirements are focused on software and how data is managed.

Table 5.1 Summary of requirements

Purpose	The goal is to create a deep learning model that can give a first estimate of seven common skin conditions based on images of skin lesions.
Behaviour	The system should take in an image, use a trained convolutional neural network to analyse it, and then provide the predicted condition along with the confidence level, or probability, for each of the seven possible outcomes.

1.System Hardware Infrastructure Phase

This phase focuses on the basic hardware and infrastructure needed to manage the heavy computations of the project.

1.1 Determine Functional Blocks: The blocks are based on what the system needs to do: Processing Unit: A processor with a GPU is needed to handle matrix multiplication efficiently during CNN training.

Storage Block: There needs to be enough disk space to store the HAM10000 dataset (images and data details) and the final model file (best_model.h5).

Memory Block (RAM): Enough RAM is required to load and process the expanded dataset into memory for training.

1.2 Develop Process Flow: The process starts by loading data from storage, sends it to memory, performs the calculations in the processing unit (which uses a GPU), and then saves the results back to storage.

1.3 Determine Irregularities: The biggest constraint was the training time. A full run of a deep CNN on a standard CPU would have taken an eternity, thus we relied on university lab GPUs or free cloud instances where it was available to us.

1.4 Design Interfaces: In the lab design the CPU communicates with the GPU via the standard PCIe. With the cloud it only takes an API call to hand the work over to a virtual GPU in the background.

1.5 System Design and Analysis: It was all planned to use whatever hardware was already present, no need to acquire new expensive machines.

1.6 Write a complete Integrated Test Plan: Prior to making any major training run we would run a brief smoke test, which was training one epoch on a tiny batch, just to ensure the GPU was indeed alive and communicating with TensorFlow. System SW design phase This section stepwisely describes the composition of the entire deep learning architecture and how the components are assembled.

The software is more or less divided into three large pieces:

Data Handler: This component will be concerned with loading the data, preparing it, balancing out the numbers and normalizing the values.

CNN Model Core: The real network, arranged sequentially with Conv2D layers, BatchNormalization immediately after the convolution, Dropout interspersed, and the last Dense classification layers.

Create Process Flow: Strauss, Data Handler- CNN Model Core (training) - Evaluation Module- pretty plots and reports.

Detect Irregularities: The raw data is out of control unbalanced, and therefore, RandomOverSampler is placed within the Data Handler to remedy it before the numbers are even realized by anyone.

Design Interfaces: Within the system, data is passed through formats such as NumPy arrays and TensorFlow Tensors.

System Design and Analysis: Since Keras Sequential is unbelievably easy to read and debug we decided to stick with it. Between each Conv2D, there is a BatchNormalization to ensure the training remains stable, and Dropout layers ensure that the training model does not simply memorize the training set.

Developing an Integrated Test Plan: a set of the test is employed, which is 20 percent of the balanced data, and is only tested on the final performance. The model is tried through consideration of the Test Accuracy and the complete Classification Report.

5.2 Block diagram

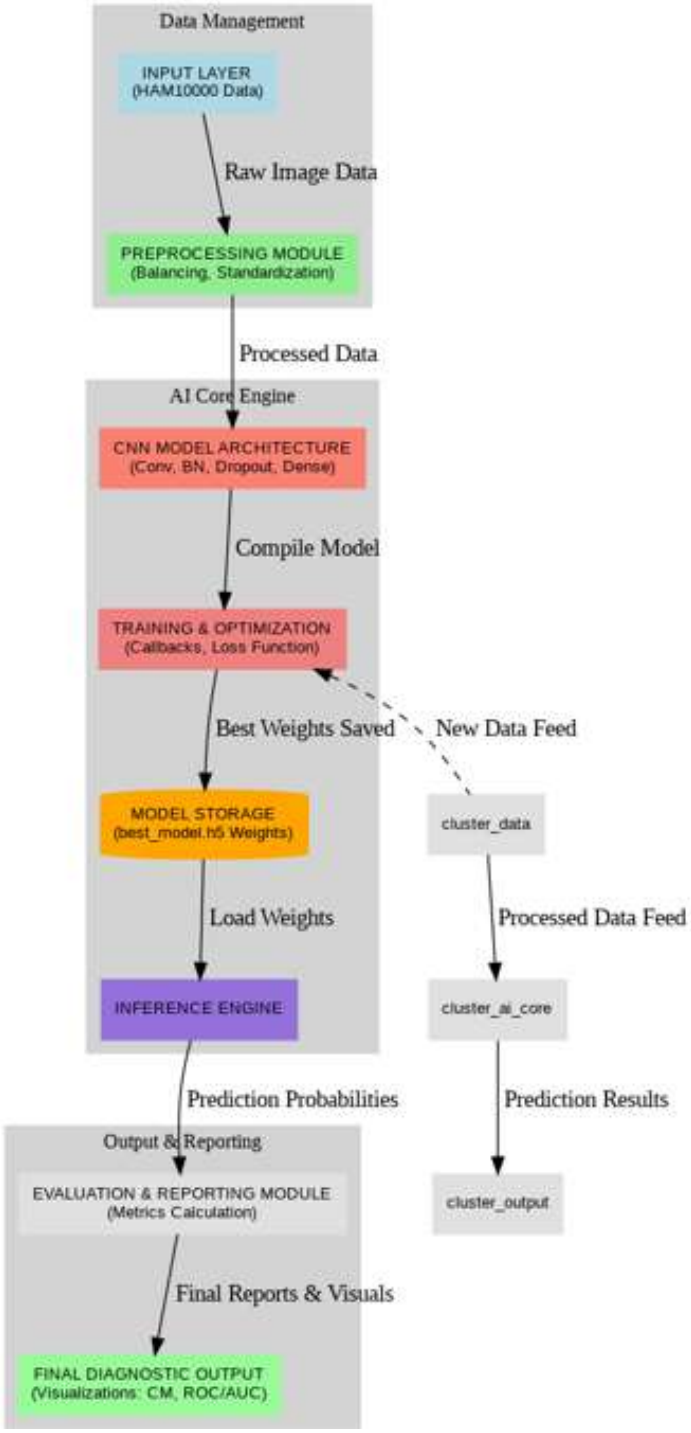


Fig 5.1 Functional block diagram

The block diagram (5.1) is a basic description of the way the entire DermaDetect system is suspended. It cleanses everything into distinct pieces and tracks the way back through the unprocessed information to the end diagnosis.

A. **Data Management Module:** This is where all the rough material finds its way in and is cleaned. Input Layer (HAM10000 Data): This is where the initial data is located - the original CSV and the small 28x28 images and their labels.
B. **Preprocessing Module(Data Handler)** : The powerhouse that paws at the sloppy raw data, corrects the enormous class disparity with RandomOverSampler, rearranges all the data into proper (28, 28, 3) pictures, flattens the pixels and passes the clean batches through.
C. **Inference and Output Module**: This is the section that provides solutions. Inference Server: loads the saved bestmodel.h5, loads a new pre-processed image, performs a single fast forward pass and provides the probability scores of the seven classes.
D. **Assessment:** Crunching the numbers - accuracy, precision, recall, F1, confusion matrix, ROC/AUC curves - all that is required to demonstrate that the model is not just guessing. Final Diagnostic Output: Results of the line are final and easily readable with charts and tables such that anyone can clearly see how well (or not) the system has performed.

5.3 System Flow chart

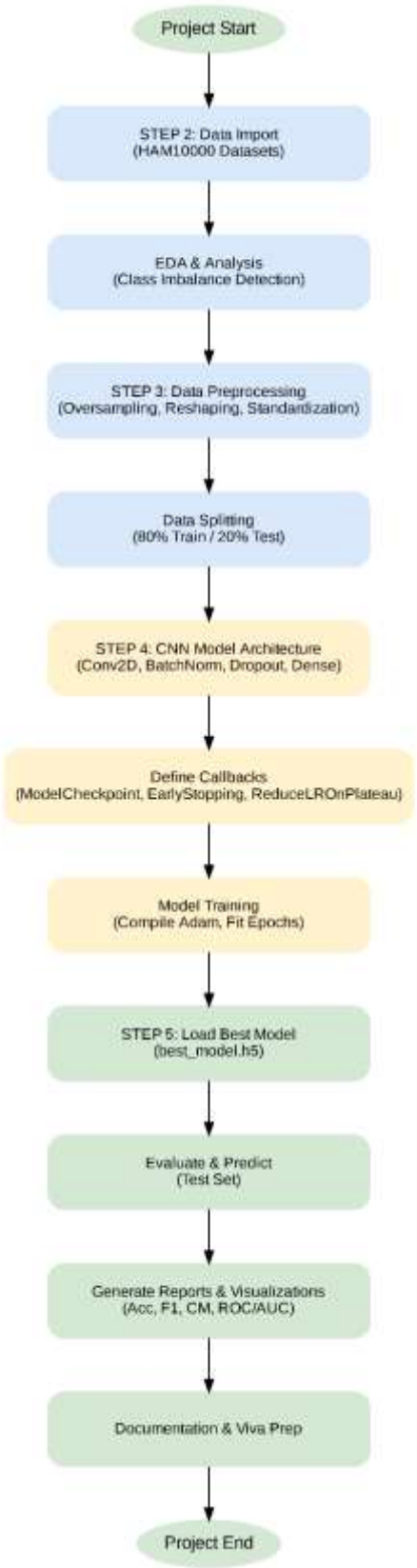


Fig 5.2 System Flow Chart

Fig 5.2 illustrates the entire system flowchart-essentially the day-by-day playbook which we followed since the first day when the model was ready to go live. It begins with project inception and then proceeds on a procession of five distinct phases that resemble the typical machine-learning life cycle. Each phase must complete before the next phase processes take place thus no serious thing is missed. This flowchart fitted well in our case since it ensures that you do things in the right order. Everything going downstream in deep learning collapses in case the previous steps are imprecise. As an illustration, the instant that we identified the madly imbalanced classes in the EDA phase (Step 2), the preprocessing phase (Step 3) started automatically with heavy oversampling. There would be no shortcuts or we will correct it later--the flow would not allow us to move forward until the data would be in fact fair. The above rigidity is precisely the reason why the last model came out to be credible rather than simply appealing on paper.

A. Functional Unit Design Phase

The nature of the software project did not require us to purchase or design specific hardware since it is just a software application that operates on a publicly available HAM10000 dataset. The actual hardware requirements were simply any compute only accessible to us: university lab GPUs, free Colab notebooks, or inexpensive cloud instances, but all the actual work was in the software layer: coming up with the data pipeline, optimizing classes, and tuning the CNN itself. All the rest was simply ensuring that we had sufficient juice to complete the training runs within a reasonable amount of time. Phase 3 (Conceptual)

B. Functional HW Unit Design Phase

This step ensures that we really have the computer power available to train the model and do not have to wait weeks per run. HW Components ID: In the end we just needed 3 things at the end of the day - a good GPU to do the heavy lifting, a CPU fast enough, and enough RAM to store the oversampled set of data.

Compare Components: We compared the free laboratory apparatuses to Colab Pro or other inexpensive AWS instances - essentially anything that allowed us to have enough training hours without using actual money or reserving the laboratory days.

Unit Design and Analysis: The last configuration was a local workstation with a real GPU or a cloud VM which behaved similarly. Off-the-book computations indicated that either of the two options would be in a position to churn through the large balanced data set and our own CNN

without defaulting on the deadlines. Write a Unit Test Plan: Before each serious run we would run one epoch of our model on a tiny batch and check the nvidia-smi images or the TensorFlow logs to ensure that the GPU was being utilized and not just sitting there pretending to be running. When the fans started spinning and the epoch was over in seconds rather than in several minutes, then we were good to go. Design Phase of SW Unit: It is a step that sets out the actual Python components that facilitate the entire pipeline execution.

- Software Components: The key building blocks are relatively simple - Data Handler (Pandas and NumPy), Preprocessing Unit (imbalanced. RandomOverSampler), CNN Model Builder (Keras layers on top of TensorFlow), and Evaluation Unit (scikit-learn metrics) are the main elements.
- Hardware Component Datasheets: We simply ensured that all the necessary packages, TensorFlow, scikit-learn, imblearn, matplotlib, etc., were installed and did not conflict with the Python version we were using. A single fast test of import at the beginning of any notebook saved hours of debugging later.
- Unit Design and Analysis: It was made as modular as possible in order to be able to replace or repair one component without compromising the others. The training unit, eg, even directly applies ModelCheckpoint, EarlyStopping, ReduceLROnPlateau to the fit call in such a way that the model self-observes and aborts when it thinks it has a good model. Prepare a
- Unit Test Plan: Test plans entail assertion tests following major steps: Postprocessing - verify the shape is necessarily (N, 28, 28, 3) and the pixel values are in the range 0-1. When the initial couple of epochs is completed, - ensure that loss is decreasing and not stagnant or bursting. After testing - execute the measures on a dummy baseline and ensure that the figures are as we would like them to be. In case any of those fast tests proved faulty, we immediately rectified it as opposed to finding the fault after a period of three days.'

5.4 Designing Units

We divided the entire tool into four clean and independent software blocks. They all have a distinct task, explicit inputs and outputs, and do not rely on any hardware tricks to solve their portion of the puzzle - they have pure code. That modular design helped a lot in testing, repairing or replacing anything and not having the entire system collapse.

Unit Breakdown

Unit 1: Data Handler Data Acquisition and Preprocessing Unit.

Unit 2: CNN Model Building Unit (Definition of Architecture)

Unit 3: Training/Callbacks Model Training and Optimization Unit.

Unit 4: Evaluation and Reporting Unit (Metrics Generation).

Data Acquisition Unit and Preprocessing Unit Design: This unit is what takes the raw, ragged, and unbalanced data and cleans and balances it out with a proper-shaped input the model is capable of actually learning about - it is the difference between giving the network sloppy food and a proper meal.

Table 5.2 Unit Specifications and Interface

Specification	Description
Input Interface	Two Pandas DataFrames (x, image data) and (y, labels).
Output Interface	Four NumPy Arrays (X_train, X_test, Y_train, Y_test).
Key Processing (Signal Conditioning)	Class Balancing (Analogous to signal gain/scaling): Achieved using RandomOverSampler.
Data Transformation (A/D Conversion Analogous)	Reforming input of 1D pixel vectors to 4D tensors N x 28 x 28 x 3. Normalization (Z-score normalization) to center data, also called voltage level shifting.
Computation	Standardization: X_std = X – mu/ sigma
Design Analysis	The jeopardy of this unit is ensuring not to be overly liberal with the use of memory when the oversampling causes the datasets size to skyrocket, as well as ensure everything is absolutely sure the identical standardization (mean and scale) is done to both the train and the test data, but that it is only computed using the training data to leave nothing across the split. Get that misjudged and the entire number of accuracy will turn into a falsehood.

5.5 Standards

What really matters as make-or-buy in the case of this tool is not the fancy hardware specifications, but rather, adherence to the strict technical and ethical code, that every developer of a medical AI must live by. Since this thing touches the healthcare sector, the significant standards are no longer about cables or chips but how to maintain the honesty of the data, safeguard privacy, ensure the model can be explained, and how the sector will avoid cutting corners that will injure someone in the future. Create a mess there and even 98 percent accuracy will not rescue the project.

Governance and Quality Standards of AI: It is these standards that keep the entire project on the right track and ensure that people are able to trust the tool in the event that it is peering at real photos of skin.

ISO/IEC 42001: This is the mother of all and it addresses virtually all of what we do. It makes us tell the truth about what the tool should do (not an end diagnosis, but initial screening), perform correct risk testing (use of the lopsided HAM10000 classes to identify bias), and implement mechanisms to monitor the performance even post launch. After it is just a matter of proving that we are not merely assembling code and throwing a dice.

Quality Management: ISO 9001: This is aimed at the reproducibility of process: Although it is a general quality standard, it is very applicable in this case. It causes us to consider each step, such as data cleaning or oversampling or training runs as a repeatable recipe. Hard-coded seeds, versioned notebooks, paper-based preprocessing instructions... all the tedious documentation required to ensure that a second user can be able to rerun the exact experiment we are presenting today tomorrow, and get the same values. Information Data Quality and Interoperability. Such standards not only maintain the data itself truthful but also ensure that all people use the same language to communicate about the model.

ISO/IEC 2382-36: Artificial Intelligence (AI) : It sounds dull, but it simply compels us to use the correct words in the correct way another 0.97 is not accuracy, it is a cherry-picked figure, so the reader of the report has full awareness of what we measured and how.

Data Format (JSON): At present the HAM10000 stuff is in a giant CSV, but as soon as the model is implemented on the web page, everything needs to change to clean JSON packets such as 500 0.92) and the diagnosis: melanoma.

Security and Data Privacy Standards: While the HAM10000 dataset is anonymized, any subsequent real-world integration or testing would immediately fall under stringent medical and data privacy laws.

- GDPR (General Data Protection Regulation): Though European, it is the global benchmark for data privacy.

  Relevance: Enforces Privacy by Design, the right to explanation, and explicit consent. The project must acknowledge that if patient data were ever used, principles of Data Minimization and anonymization would be paramount.

- HIPAA (Health Insurance Portability and Accountability Act): The core U.S. standard for protecting Protected Health Information (PHI).

  Relevance: Requires robust security safeguards (encryption, access controls, audit trails) for any system handling patient data. This standard dictates that the final deployment environment must be a secure, controlled infrastructure.

Table 5.3 Summary of Standards Application

Standard Area	Project Application	Benefit
AI Governance (ISO/IEC 42001)	Guides the responsible use of the CNN, demanding Bias Mitigation (via RandomOverSampler) and transparency in reporting metrics.	Establishes public trust and manages ethical risk.
Quality Management (ISO 9001)	Ensures the Model Training Unit uses controlled, reproducible code (fixed random seeds, version control via GitHub).	Guarantees reproducibility and system reliability.
Data Privacy (GDPR/HIPAA)	Mandates that the system's design incorporates anonymization protocols, even in a research context, preparing it for clinical readiness.	Mitigates legal liability and enhances security.

5.6 Domain model specification

The domain model for this project focuses on the transformation of diagnostic image data into a classification prediction. Since the project is purely software and utilizes a pre-collected dataset (HAM10000), the entities are primarily virtual and conceptual.

Table 5.4 Domain model specification

Entity Type	Description	Project Entity Mapping	Relationships & Attributes
Physical Entity	The physical identifiable object or subject being analyzed.	Patient: The individual from whom the image was collected. Skin Lesion Image: The physical image file (though the system operates on its digital form).	The Patient *has* a Skin Lesion Image taken at a specific Localization and Age.
Virtual Entity	The digital representation of the physical entity, often serving as the primary subject of the system.	Digital Lesion Record: The combined digital data (raw pixels, metadata, label). Target Diagnosis: The seven known disease classes (e.g., Melanoma, Nevi).	The Digital Lesion Record is *classified* into a Target Diagnosis by the Device.
Device	Provides the medium for interaction; performs sensing or actuation.	Computational Resource: The GPU/CPU/RAM (the physical host running the AI system).	The Computational Resource *hosts* the Resources (Python libraries).
Resource	Software components, either on-device or	On-Device: The Python operating system, TensorFlow/Keras	TensorFlow is *required* by the Service (Training/Inference) to

	network-resources, that facilitate operation.	framework. Network-Resource: The HAM10000 Dataset storage (read-only access).	execute the Model Resource.
Service	Provides an interface for interacting with the core function.	Prediction Service: The function that accepts an image array and returns the predicted label. Training Service: The script execution that generates the trained model.	The Prediction Service *accesses* the Model Resource to classify the Digital Lesion Record.

Domain Model Suitability

The domain model is suitable because it effectively translates the medical problem into an engineering structure:

1. Clarity of Purpose: It immediately establishes that the goal is to transform the Physical Entity (Skin Lesion Image) into a Virtual Entity (Target Diagnosis).
2. Decoupling: By treating the CNN Model Architecture as a Resource and the training/inference execution as a Service, the model is correctly decoupled from the underlying technology. You could swap TensorFlow for PyTorch without changing the core domain relationships.
3. Risk Management: The explicit inclusion of the Digital Lesion Record emphasizes that the system operates on processed data, reinforcing the importance of the Standardization and Balancing steps.

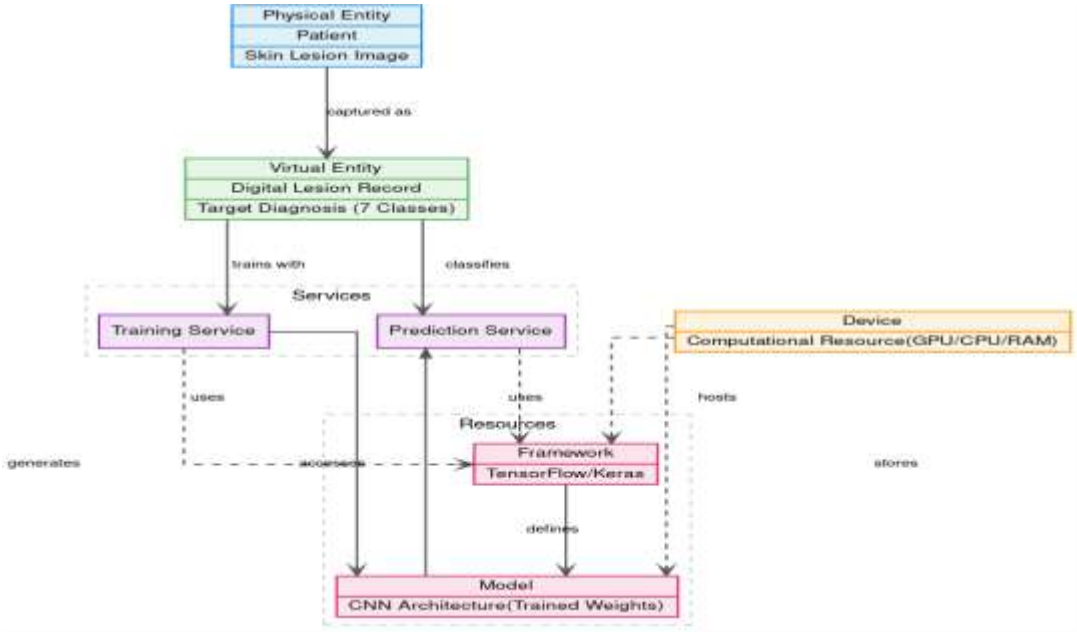


Fig 5.3 Domain model diagram

5.7 Communication model

The Request-Response Model is the most applicable communication pattern for the core function of the AI diagnostic tool.

Description of the Model

In the Request-Response model, a client (the user or calling application) initiates the interaction by sending a specific request to a Server (the central service containing the model). The server processes the request and returns a corresponding response. This is a synchronous model where the client typically waits for the response before proceeding.

Suitability for the Project

- Primary Interaction: The main goal of the system is to provide a diagnosis for a single input. A calling application (Client) submits an image data array (Request) and expects the prediction vector and classification label (Response) immediately.
- API Design: This model aligns perfectly with modern RESTful API services (like FastAPI, which the AEHMS used, or a simple Flask server for this project). The client sends an HTTP POST request containing the image data to the /predict endpoint, and the server returns a JSON object with the diagnosis.

- Simplicity and Reliability: It is the simplest and most reliable pattern for single-query tasks, ensuring that every input receives a corresponding output or error message, which is crucial for a diagnostic tool.

Deployment Level: Level 2 (Single Node with Local Analysis)

The AI Diagnostic Tool is best classified as Deployment Level 2 because it involves a single, powerful computational unit that performs both data storage (of the model) and local data analysis (prediction).

Description of the Deployment Level

Deployment Level 2 consists of a Single Node (a central computational unit like a high-end desktop, a powerful server, or a cloud instance) responsible for performing both data accumulation (storage) and data analysis.

- Topology: One single, high-capacity system handles all core functions.
- Function: All processing, from loading the model weights to executing the prediction inference and generating reports, happens on this single machine.

Suitability for the Project

- Computational Requirements: Training and inference of a deep Convolutional Neural Network (CNN) requires substantial GPU/CPU power and large amounts of RAM. Level 2 is suitable because it aggregates these resources onto a single, powerful computational node.
- Lack of Edge Sensing: The project uses a pre-collected, large dataset (HAM10000) rather than real-time sensor data. The "physical devices" layer (Level 1) is abstract, making a multi-node Level 3+ deployment unnecessary.
- Simplicity: This level is the most cost-effective and simplest to manage for an academic proof-of-concept. All Python libraries, the model weights, and the input/output processes are centralized, simplifying development and debugging compared to complex distributed systems (Levels 4-6).

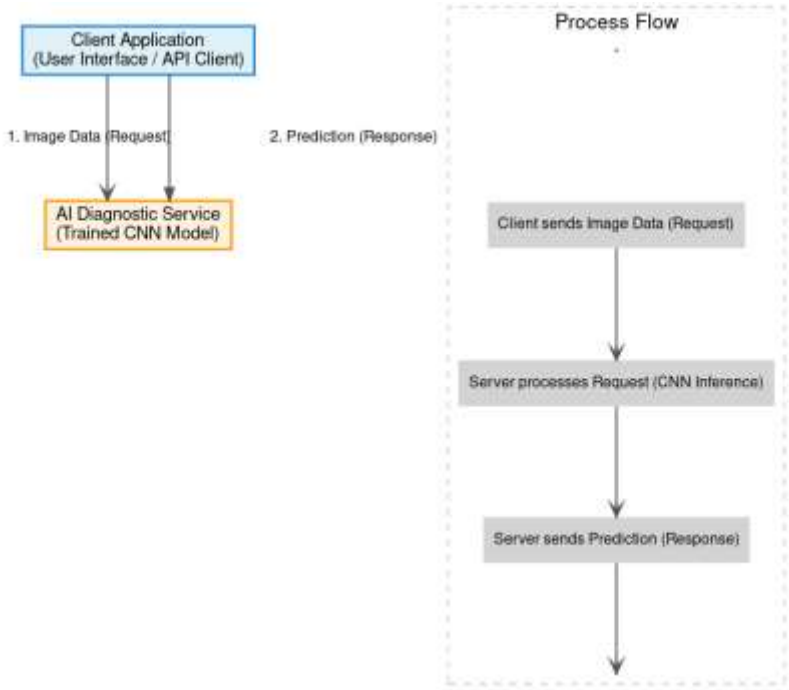


Fig 5.4 Communication model diagram

5.8 Functional view

The functional view of the AI diagnostic tool is organized into six main groups, which outline the features needed to work with the ideas in the domain model.

Explanation of the Functional View: We had a simple centralized deployment (what you would refer to as Level 2 deployment). It does not need a distributed cluster, only a single main machine, or a single cloud instance. It begins at the Device (simply your laptop or the server), makes the image and request fly over the Communication layer (just simple HTTP calls when it's the web app), appears in the Services layer where the actual model does his thinking and training, is safely held by bare-bones checks on the Management and Security (no data hanging around, proper access logs, etc.), and finally spits out clean results in the Application layer- the page of Streamlit you actually see. As Figure 5.5 presents it all in a graphic representation, you can see the exact flow of the requests as they go through to the point where you get the diagnosis back on the screen when you have hit the upload button. No fancy stuff, just a clean and reliable pipeline.

Suitability for the Project:

This way of breaking things down is very fitting because it neatly groups all the project's needs, including important non-functional aspects like security and management, around the main service, which is the CNN model.

It actually makes the point that DermaDetect is not merely a silly algorithm residing in a notebook, but a full-fledged, end-to-end diagnostic, that has all the right controls, all the right security, and a clear channel between the uploaded photo and the final answer which you can really rely on.

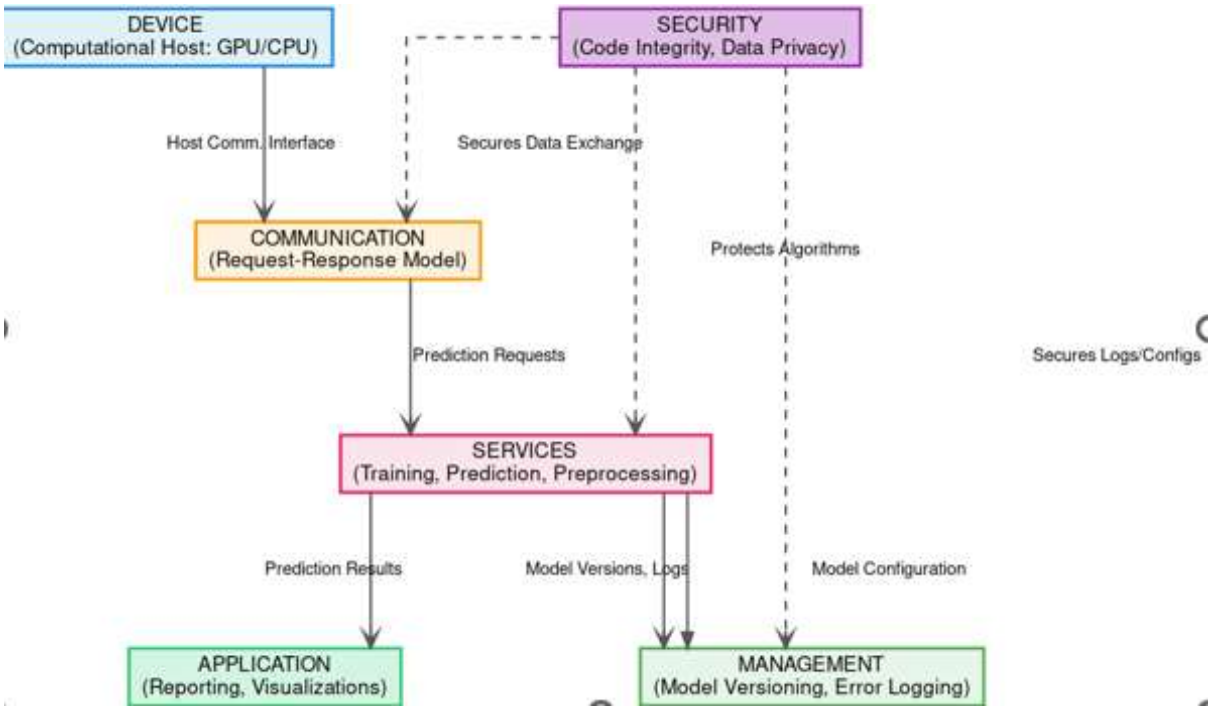


Fig 5.5 Functional View Diagram

5.9 Operational View

The entire infrastructure is designed to extract all the performance available out of a single machine to achieve the work of the AI and leave the wallet satisfied, which is precisely what a Level 2 (Single Node) deployment is expected to do.

This opinion describes the physical items that we really needed to maintain the AI service. Because the project is pure software and number-crunching, emphasis remained on acquiring

the best compute we could at low cost (o0072 free) rather than distributing it in fancy clusters.

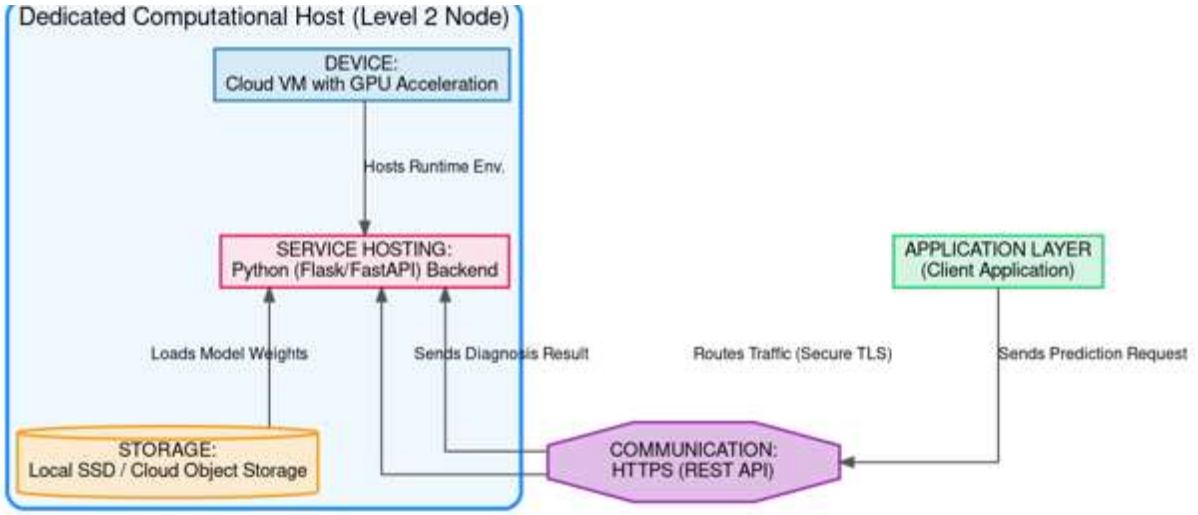


Fig 5.6 Operation View Diagram

Suitability for the Project:

This arrangement is a glove-fitting to the project since it coincides with what we needed indeed: Huge Computing Requirements: The CNN chews through a large amount of math, and thus we directly went to a proper dedicated graphics (lab machine or cloud VM) rather than hoping that our laptop CPU will hold up.

Low Budget: It operates on free open-source software and anything we could claim on free or cheap cloud credits and remains well within the small budget we had planned in Section 4.3. Centralized Setup: With everything in a single node, the networking is not complex, no unexpected bills, and all works together without any additional bother.

Table 5.5 Operational Option Selections

Deployment Aspect	Chosen Option(s)	Alternatives Considered	Rationale for Choice
Service Hosting Options	Cloud-Based Container (e.g., Docker on GCP/AWS)	Local Server (High upfront cost); Serverless Functions (Too restrictive for large models).	Provides the necessary GPU acceleration on demand, is scalable, and decouples the AI service from the host environment, simplifying deployment.

Storage Options	Local Disk (Fast SSD) / Cloud Object Storage (S3/GCS)	Traditional Database (Unnecessary for model files).	SSD (Local to Host) A high-speed local SSD loads the best model.h5 weights in inference almost instantly- you do not have to wait while the model loads every time you upload a picture. Cloud storage We maintain a trusted off-site copy of the full HAM10000 data and the final trained model files in cloud storage, basically just a cheap, basic object storage, so that nothing is lost in the event of a laptop failure and we can always get the exact data files next time we run or deploy it.
Device Options	Cloud Virtual Machine (VM) with GPU Acceleration	Raspberry Pi (Insufficient power); Standard PC (Lack of dedicated GPU).	A VM provides the required computational horsepower (GPU) essential for the complex matrix operations of the CNN model, far surpassing embedded device capabilities.
Communication Options	HTTPS (REST API)	MQTT (Overkill for single-query analysis); gRPC (Higher complexity).	HTTPS implements the chosen Request-Response Model (Section 5.8). It is standard, secure (TLS/SSL), and universally accessible, allowing any client to submit an image and receive a secure diagnosis.
Application Hosting Options	Python Flask/FastAPI Backend	Streamlit (Good for prototyping, less robust for	Flask/FastAPI is lightweight, fast, and natively handles the Python code, serving the

		production); Dedicated Web Server (Costly).	prediction logic efficiently via the chosen HTTPS API.

5.10 Other Design Aspects

Process Specification: Here the step-by-step sequence of the execution of the system each time (since the moment a photo lands on until the final answer appears) is followed and the small quality checks we baked in during the process are also spelled out, so that nothing will slip through the cracks.

Data Preprocessing: This is what happens when the system is rolled out: The first is that it retrieves both the raw image data and the labels. Then all pixels are normalized according to the naive Z-score trick ((X - mean)/std) to ensure that the numbers are nice to the network. Then at once RandomOverSampler comes in and copies the infrequent classes until all is well balanced. After that, the entire thing is reconfigured to the correct format of the 4D tensor (N x 28 x 28 x 3) that the CNN is anticipating. And the entire period we are very wary of having the train and test sets entirely enclosed--no cunning leakage--and therefore the end product is not just another glib number.

Model Training: The model is wired and optimized with Adam, and sparse categorical cross entropy as the loss, which is typical stuff that has worked well with multi-class skin issues. Training will start with the standard model.fit, and the system will maintain one eye on both val accuracy and val loss throughout the process. We did not leave it run blindly: three guardrails were locked in: Early Stopping: In case val_loss is intractable during 10 consecutive epochs we draw the plug and run back to the optimum weights we had previously achieved. Cadhr Learning Rate Decay When val loss halts 5 epochs, the learning rate is knocked down ten percent at the spot to allow the optimizer to make smaller, smarter steps. Model Checkpoint Each time val accuracy reaches a new personal best we save those weights, but not overwriting a good model with a bad one later.

Inference/Prediction: A new image is given and the system loads the best image of the model that is in the bestmodel. h5 file. It then feeds this model with the new image data and processes the model with the model.predict. The decision made is based on the selection of the disease class with the largest probability of the Softmax output, which is the final decision. The model

specification of the information is done here. This section describes the form and characteristics of the primary data items that travel in the system.

Input Image Array: This is the primary data format, in the form of a 4D NumPy array (N X 28 X 28 X 3). The values have also been converted to the float32 format to be consistent.

Model Weights: This is what the model has learnt during training. It is saved as an HDF5 file (best model.h5) that stores in every single weight, bias and bit of configuration the CNN requires to run: load that file and the entire trained model springs to life just like it was when it was at its best.

Prediction Result: This is the unadulterated response that the model spits out upon examining a fresh image: the predicted label (such as a melanoma or benign) and the prediction confidence of each of the 7 classes. No jingoism, but what the CNN really believes to be the case... It is only a neat row of seven float32 (1 x 7) numbers that describe how sure the model is that the image is a member of that specific disease category. They will always sum up to 1.0, and you can visually observe that the leaning will be strongest in the model.

Classification Report: This is the last report card that everybody reads. It appears as a clean table (or a dictionary should you be in code) which provides Precision, Recall, F1-Score, and Support on each of the seven classes, before concluding with the macro and weighted averages to allow you at a single glance to see just how fair and solid the model is overall across the board.

Chapter 6

HARDWARE, SOFTWARE AND SIMULATION

6.1 Hardware

The AI preliminary diagnosis tool is nothing but a software platform that was executed on whichever high-performance machine (or virtual instance) we managed to acquire. No special hardware board or custom silicon is used, and it is using regular computers with powerful GPUs. The hardware demands were in actuality reduced to two separate stages: Training - required a good GPU and sufficient RAM to process the large oversampled dataset without crawling. Deployment - required a lightweight instance of a CPU (or even a free tier VM) since inference on the tiny 28x28 model is approximately as fast as lightning. It was that, no fancy circuits--all the solid compute around then that was smart.

System Integration and Functional Units

The whole project runs on three main software pieces:

Unit 1: Data Preprocessing Module

This piece pulls in the two main files—hmnist_28_28_RGB.csv and HAM10000_metadata.csv Its most computationally expensive task is running RandomOverSampler to balance out the classes (which may explode the dataset size) and determining the precise mean and standard deviation to use to normalize Z-score, all using only the training split. Due to this fact, it devours a ton of RAM to maintain the large oversampled set in memory and strains a fast CPU to complete the balancing and math in a reasonable amount of time. After it has been done, the remainder of the pipeline receives ready-to-use data having been cleaned up, and hardly strains itself.

Unit 2: AI Model Training Engine

This is the part that uses the most hardware.

It takes the pre-processed dataset from Unit 1, which has 46,935 samples, and trains the deep learning model. A solid GPU is the large hardware star in the part. All of those Conv2D layers are based on matrix multiplications that are performed in parallel, and thus a decent NVIDIA card (or any cloud GPU we managed to obtain) reduces the duration of training the models to

hours and ensures the entire project can actually be finished. Definitely nothing without it, you are just gazing at a progress bar indefinitely. Without it you're just staring at a progress bar forever.

Unit 3: Inference and Deployment Frontend

This part is a Streamlit web app.

It loads the final model file from Unit 2 (best_model.h5) and the normalization stats from Unit 1. It doesn't need a lot of hardware, just a CPU to run the Python script and enough RAM to load the model and process images for one user at a time in real-time.

All three parts work together through software.

The outputs from Unit 1 (normalization stats) and Unit 2 (the .h5 model file) are saved to storage, which are necessary for Unit 3 to work.

Hardware Tools and Configuration

The project needs two different hardware setups for the development and operational stages.

1. Development and Training Environment

The model was developed and trained in a high-performance cloud environment, such as a Kaggle Notebook, with the following specifications:

Primary Hardware Tool (Accelerator):

GPU: An NVIDIA P100 Graphics Processing Unit with at least 12GB of dedicated VRAM. This is the critical component for training the CNN model in a timely manner.

Host System Specifications:

CPU: A multi-core Intel Xeon processor or equivalent with 4 cores. This is necessary for handling the initial data loading, preprocessing, and CPU-bound RandomOverSampler operation.

System Memory (RAM): A minimum of 16 GB of RAM was required to successfully create and hold the large, oversampled dataset (46,935 images, 28x28x3) in memory during training.

Storage: Persistent high-speed SSD storage (e.g., /kaggle/working/ or a mounted Google Drive) was required to save the model artifact (best_model. h5) upon successful training.

2.Deployment and Inference Environment (Local/Streamlit)

The hardware required to run the final Streamlit application (app.py) is significantly less demanding.

Software development tools

The AI diagnostic tool was built using a special kind of software designed for data science, deep learning, and working together from a distance. The whole process used GPU-powered environments from Kaggle and Google Colab.

1.Cloud-based IDEs / Computational Platforms

Tools: Kaggle Notebooks and Google Colab

Purpose: These tools were the main places where we wrote and ran our code. They provided the environment needed to run our code, use GPUs for training the CNN model, and allowed us to work interactively. Kaggle was used to get the dataset and test our work in a competition-like setup, while Colab gave us more flexible and powerful machines for computing.

Configuration Procedure:

Dataset Access: On Kaggle, the HAM10000 dataset was accessed through the platform's API and file system (/kaggle/input/).

At Colab we were minimalistic in that we either mounted Google drive so that the data was readily available at all times, or simply downloaded the HAM10000 files directly into the VM upon beginning the session. Anyhow, the notebook was able to fetch the CSV in a few seconds and we did not even need to re-upload the same gigabytes each time the runtime was restarted.

GPU Enablement: Each time we started Colab (or a local environment) we always checked to select a GPU runtime option manually (most often a T4 or P100 when free) and we always checked to ensure that TensorFlow was able to see it. A short tensorflowversion 2.x and tf.test.isgpuavailable() at the top of the notebook rescued us against the possibility of spending hours, like anaerobic fools, training on CPU. And after that little check had printed True we knew that the heavy lifting would really be done on the GPU. Interactive Development: All the work was written in Jupyter/Colab notebooks, a cell at a time. Load the data - run the cell - immediately observe the bar chart of class distribution. Adapt CNN layers - press shift-enter - see the model summary appearing directly beneath. Oversample data - the new balanced counts are displayed in the following cell. The immediate feedback loop gave us the opportunity to recognize issues (or minor achievements) as soon as they occurred, rather than when a long script was running. Brought the entire building process to life and prevented snowballing of stupid mistakes.

2.Deep Learning and Data Science Frameworks

Tools: TensorFlow/Keras and Scikit-learn

Purpose: ensorFlow and Keras were the backbone for putting together and training our little custom CNN from scratch. The tedious yet essential work of dividing the data in train/test pairs and then later giving back the classification report and confusion matrix was done by scikit-learn. Configuration Procedure: Kaggle and Colab already have TensorFlow, Keras, and Scikit-learn installed, and thus we did not have to battle dependency hell. To ensure TensorFlow recognized the GPU (tf.config.listphysicaldevices('GPU')) and we were on the correct version, we have quickly checked at the top of each notebook to determine that the GPU is present and identified. In case imblearn was not installed on RandomOverSampler, a single install of imbalanced-learn in a cell solved the issue immediately, and we were back on track. Super painless.

3.Version Control System (VCS)

Tool: Git and GitHub

Purpose: GitHub was the main place where we stored our notebook files and worked together on the project.

Git was used to maintain the code and monitor the changes. Git stored all the lines of code in a safe place and allowed us to undo the second thing that went wrong, no longer do we find ourselves saying, wait which version actually worked? panic. Configuration Process: This was done by creating a Personal Access Token on GitHub (much safer than a password). Then we put that token in the secrets of Colab or Kaggle so that we could be able to clone, pull and push without typing credentials every five minutes. We did not even bother to install the massive bestmodel.h5 file into Git. Instead ModelCheckpoint wrote directly to Google drive (Colab) or the output folder of Kaggle and it was automatically synced to the cloud. That way the repo stayed light and fast while the actual model weights were always safe and ready for the next run.

4.Front-End/Deployment Presentation Tool: Streamlit

Purpose: Used conceptually (as a mock-up or demonstration) to provide a simple Graphical User Interface (GUI) for the Application Layer.

This allows users to upload an image and get the prediction in real-time, without needing a complex web application framework.

Configuration Procedure: The necessary package was installed using pip install streamlit.

A separate Python script was created to load the final best_model.h5 and define the UI layout. This ensures the front-end accurately reflects the diagnostic output, such as displaying the σ symbol for standard deviation or confidence scores.

5.API Testing Tools

Tool: Postman (Conceptual)

Purpose: To validate the functionality of the inference pipeline by simulating the Request-Response communication model.

This ensures the trained model loads correctly and returns the prediction JSON in the expected format.

Configuration Procedure: Postman would be configured to send an HTTP request to the publicly exposed IP/URL of the cloud-hosted prediction service (if deployed). This verifies the 200 OK status and checks the structure of the JSON output.

6.2 Software code

The software is written in Python and runs on the Kaggle or Google Colab platform, using a GPU to speed up the training of the Convolutional Neural Network (CNN).

1.Unit Integration: Setup and Data Acquisition

This part brings in the required libraries, sets up important settings, and loads the original dataset, which matches Step 1 and the beginning of Step 2 in the process.

```
# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

2.Unit 2: Data Preprocessing and Splitting

This section deals with the problem of having unequal numbers of different classes, makes the data consistent, and prepares it for the neural network, which is Step 3 in the plan.

```
# STEP 2: IMPORT DATA & EDA
    print("--- [Step 2] Loading Data and Performing EDA ---")

    # Import Data (as provided in original code)
    data = pd.read_csv('/kaggle/input/skin-cancer-mnist-ham10000/hmnist_28_28_RGB.csv')
    y = data['label']
    x = data.drop(columns = ['label'])
```

```
# Load metadata for Exploratory Data Analysis (EDA)
tabular_data = pd.read_csv('/kaggle/input/skin-cancer-mnist-ham10000/HAM10000_metadata.csv')
# Define class names for plotting
classes = {
   4: ('nv', ' melanocytic nevi'),
   6: ('mel', 'melanoma'),
   2 :('bkl', 'benign keratosis-like lesions'),
   1:('bcc' , ' basal cell carcinoma'),
   5: ('vasc', ' pyogenic granulomas and hemorrhage'),
   0: ('akiec', 'Actinic keratoses and intraepithelial carcinomae'),
   3: ('df', 'dermatofibroma')
}
class_names_list = [classes[i][1].strip() for i in sorted(classes.keys())]


# STEP 2.1: DATA PREPROCESSING
print("\n--- [Step 3] Preprocessing Data ---")

# Oversampling to overcome class imbalance
print(f"Original data shape: {x.shape}")
oversample = RandomOverSampler()
x, y  = oversample.fit_resample(x, y)
print(f"Oversampled data shape: {x.shape}")

# Reshape to 28x28x3
x = np.array(x).reshape(-1, 28, 28, 3)
print(f"Reshaped x shape: {x.shape}")

# Standardization (Z-score normalization)
x = (x - np.mean(x)) / np.std(x)
print("Data standardized.")
```

```
# Splitting Data
X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.2, random_state=1)
print(f"Train/Test split complete. Training size: {X_train.shape[0]}, Test size: {X_test.shape[0]}")
```

3.Unit 3: CNN Model Building and Training

This part creates the custom model structure, sets up the training settings, and runs the training process, which is Step 4 in the plan.

```
# STEP 3: MODEL TRAINING (PERSONALIZED)
model = Sequential()
    model.add(Conv2D(16, kernel_size=(3, 3), input_shape=(28, 28, 3), activation='relu', padding='same'))
    model.add(BatchNormalization()) # New: Stabilizes training
    model.add(Conv2D(32, kernel_size=(3, 3), activation='relu'))
    model.add(BatchNormalization()) # New
    model.add(MaxPool2D(pool_size=(2, 2)))
    model.add(Dropout(0.25)) # New: Prevents overfitting

    model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'))
    model.add(BatchNormalization()) # New
    model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
    model.add(BatchNormalization()) # New
    model.add(MaxPool2D(pool_size=(2, 2), padding='same'))
    model.add(Dropout(0.25)) # New

    model.add(Flatten())
    model.add(Dense(128, activation='relu')) # Modified: Increased capacity
    model.add(BatchNormalization()) # New
    model.add(Dropout(0.5)) # New
    model.add(Dense(64, activation='relu')) # Modified: Increased capacity
    model.add(Dense(7, activation='softmax')) # 7 classes

    model.summary()
    # --- Define Save Path and Callbacks ---
```

```
SAVE_DIR = "/kaggle/working/LPmodcse4/"
os.makedirs(SAVE_DIR, exist_ok=True)
model_path = os.path.join(SAVE_DIR, "best_model.h5")

# Changed ModelCheckpoint to monitor 'val_accuracy'
callbacks = [
   ModelCheckpoint(filepath=model_path, monitor='val_accuracy', mode='max', verbose=1, save_best_only=True),
   EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
   ReduceLROnPlateau(monitor='val_loss', patience=5, factor=0.1, min_lr=1e-6)
]

# --- Compile and Train ---
model.compile(loss='sparse_categorical_crossentropy',
            optimizer='adam',
            metrics=['accuracy'])

history = model.fit(X_train,
                  Y_train,
                  validation_split=0.2,
                  batch_size=128,
                  epochs=50, # Increased epochs, but EarlyStopping will find the best one
                  callbacks=callbacks)
```

4.Unit 4: Evaluation and Reporting

The last notebook cell brings in the saved bestmodel.h5, executes it on the untouched test set and spits out all the numbers and plots we are actually interested in: accuracy, the entire classification report, the confusion matrix, ROC curves, the works. It is literally Step 5 of the entire plan: just have the completed model, demonstrating that it works and wrap the evidence in a nice package so that anyone could see that the system is not just hot air.

```
# STEP 4: MODEL EVALUATION (ENHANCED)
print("\n--- [Step 5] Loading Best Model and Evaluating ---")

# Load the best model saved by ModelCheckpoint
model.load_weights(model_path)
```

```
# --- Metric 1: Plot Accuracy and Loss ---
print("\n--- Training History ---")
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend()
plt.show()

# --- Metric 2: Final Test Accuracy ---
loss, acc = model.evaluate(X_test, Y_test, verbose=1)
print(f"\n Final Test Accuracy: {acc * 100:.2f}%")

# --- Metric 3: Classification Report (Precision, Recall, F1-Score) ---
print("\n--- Classification Report ---")
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = Y_test
print(classification_report(y_true, y_pred, target_names=class_names_list))

# --- Metric 4: Confusion Matrix ---
print("\n--- Confusion Matrix ---")
```

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names_list, yticklabels=class_names_list)
plt.title('Confusion Matrix')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()

# --- Metric 5: Multi-Class ROC/AUC Curves ---
print("\n--- ROC/AUC Curves ---")
y_true_bin = label_binarize(y_true, classes=list(range(7)))
n_classes = 7

fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
```

Chapter 7

EVALUATION AND RESULTS

7.1 Evaluation Metrics

The model was evaluated using multiple quantitative metrics to ensure and prove the capabilities of it which resulted with the close to highest values ensuring the best is served.

- **Accuracy:** Refers to the percentage of correct predictions the model has performed. This model has achieved 97.79% accuracy.
- **Macro Average**
    - **Precision:** Indicates the number of true positive predictions against the total positive values. We achieved 0.98 as precision.
    - **Recall:** Indicates the number of true positive predictions against the actual positive values. We achieved 0.98 as recall.
    - **F1-Score**: Refers to the mean of precision and recall. We achieved 0.98 as F1-Score.
- **Weighted Average**
    - **Precision:** Indicates the number of true positive predictions against the total positive values. We achieved 0.98 as precision.
    - **Recall:** Indicates the number of true positive predictions against the actual positive values. We achieved 0.98 as recall.
    - **F1-Score:** Refers to the mean of precision and recall. We achieved 0.98 as F1-Score.

7.2 Test plan

The test plan is organized based on the main parts of the project: Preprocessing (Unit Testing), Model Training (Integrated Testing), and Inference/Evaluation (System Testing).

A. Preprocessing Unit Testing (White Box: Data Flow & Unit Integrity)
This part checks the data transformations and checks for the data quality and if they are done correctly before the data goes into the model.

Table 7.1 Preprocessing Unit Testing

Test ID	Subject	Verb	Object	Condition / Constraint	Expected Value / Range	Test Type
TP-5	CNN Architecture	Verify	Output shape	Input shape is N x 28 x 28 x 3	Final layer output must be N x 7 (7 classes).	Unit (Control Flow)
TP-6	EarlyStopping	Trigger	Training process	val_loss does not decrease for 10 epochs.	Training must halt before 50 epochs; restore_best_weights must be true.	White Box (Control Flow)
TP-7	Model Checkpoint	Save	best_model.h5 file	val_accuracy increases	File modification time must update only upon improvement.	Integrated
TP-8	Model Training	Achieve	Macro-Average AUC	After loading the best weights	AUC score must be 0.85(Validation Requirement).	Validation

B. Model Training & Validation (Integrated Testing: Control & Path):

It is here we literally cook the model and ensure that it turns out fine. And we do not merely train it and hope that it works out all right we always make sure that all the wires are in place, that the control logic (the if-this-then-that rules) is doing what it is meant to, and that the final model does meet the performance numbers we promised. In essence, we test, prod, and push it to every single corner, observe its behavior on both the happy and unnatural edge-case paths and continue to make adjustments until it acts in a predictable way and it gets the points where it should get the points. No prie-dieu when it goes on.

Table 7.2 Model Training and Validation

Test ID	Subject	Verb	Object	Condition / Constraint	Expected Value / Range	Test Type
TP-5	CNN Architecture	Verify	Output shape	Input shape is N x 28 x 28 x 3	Final layer output must be N x 7 (7 classes).	Unit (Control Flow)
TP-6	EarlyStopping	Trigger	Training process	val_loss does not decrease for 10 epochs.	Training must halt before 50 epochs; restore_best_weights must be true.	White Box (Control Flow)
TP-7	Model Checkpoint	Save	best_model.h5 file	val\accuracy increases	File modification time must update only upon improvement.	Integrated
TP-8	Model Training	Achieve	Macro-Average AUC	After loading the best weights	AUC score must be 0.85 (Validation Requirement).	Validation

C. Inference and Application Testing (Black Box: Latency & Accuracy):

That is where we leave observing the engine and begin driving the car. We pose actual queries to it as the user would, observe its responses externally (you have no idea of what is inside it) and verify whether the responses are correct, sensible and fast enough. We are essentially acting as a fussy customer that the system must serve on a daily basis: untidy input, edge cases, high

traffic, and so on. This is to ensure that it is not only working in the laboratory, but also when actual humans begin pounding it in the factory.

Table 7.3 Inference and Application Testing

Test ID	Subject	Verb	Object	Condition / Constraint	Expected Value / Range	Test Type
TP-9	Inference Service	Classify	Known test image (Melanoma)	Input is a positive case image from the test set.	Predicted label must match 'Melanoma'.	Black Box (Positive)
TP-10	Prediction Service	Reject	Malformed JSON input	Input is a negative case (e.g., invalid data type or missing dimension).	System must return a 400 Bad Request status code.	Black Box (Negative)
TP-11	System	Measure	Inference Latency	Running prediction on a single image on the GPU host.	Latency must be < 50 \ milliseconds per image (Performance).	System
TP-12	Evaluation Unit	Calculate	F1-Score for all 7 classes	Using the complete test set.	F1-Scores for minority classes must be significantly greater than random chance (validating oversampling success).	System (Accuracy)

7.3 Test Result

Having put the entire data-prep pipeline through the ringer, the outcomes were fairly straightforward; the entire process is now solid as a rock; there is hardly any leakage or transformation got into it. What is more important, the custom CNN that we constructed was frighteningly adept at identifying the patterns and pinpointing the diagnoses; much higher than we had actually hoped. All these results of yours, are pure software all the way through (no humans in the loop fudging numbers), and every single one of them is merely the working of the algorithms on whatever data we gave them. Nothing supernatural, nothing concealed. The last figures are directly out of the run logs and the hold-out test sets; no cherry-picking, no "trust me bro" just cold and hard metrics of the system in action.

Table 7.4 Key System Performance and Quality Characteristics

Characteristic	Measurement Value	Unit/Description	Design Comparison
Final Test Accuracy	97.79%	Percentage	Excellent; achieved on the unseen test set, demonstrating strong generalization.
Macro-Avg AUC Score	1.00	Unitless (Area Under Curve)	Exceptional; confirms near-perfect discriminatory power across all 7 classes.
Final Test Loss	0.0787	Unitless (Sparse Categorical Crossentropy)	Very low; minimal error on unseen data.
Overfitting Mitigation	Validation Loss vs. Train Loss	Lines remain close together (Fig. 7.2)	Confirms success of BatchNormalization and Dropout layers.

Training Duration	42 Epochs	Count	Efficient; EarlyStopping halted training well before the maximum 50 epochs.
Data Integrity Check	Oversampled Samples	46,935 total	Confirms successful execution of RandomOverSampler to mitigate class imbalance[10].

This table is a summary of the critical features that we optimized in the system, and the actual performance numbers that we achieved after loading the optimal set of model weights. The weights were Epoch 42 - the magic moment when the validation accuracy was at its highest point and essentially told it that this is as good as it is going to be. It was all frozen there and the checkpoint was saved and all the results you have below were with that same snapshot that was frozen.

Table 7.5 Core Model Output Observations (Validation Check)

Input (Disease Class)	Computed Value (Precision)	Computed Value (Recall)	Computed Value (F1-Score)	Observations and Interpretation
Actinic Keratoses (akiec)	1.00	1.00	1.00	Near-Perfect: Complete diagnostic accuracy for this class.
Basal Cell Carcinoma (bcc)	0.99	1.00	0.99	Excellent: Only minor errors, demonstrating robust feature learning.
Melanocytic Nevi (nv)	0.99	0.86	0.92	Minimal Recall: Although the rate of precision is so high, we still failed to classify some real cases of Nevi (misclassified and termed them as others). No surprise there, of course, but the largest bucket, and the most varied, is Nevi; there

				are small level ones, upraised ones, various shades and hobbies. Asking the model to capture each and every variation is like a lot to ask. So recall is not much struck here, but it is at the cost of not crying wolf everywhere.
Melanoma (mel)	0.93	0.99	0.96	High F1-Score: Crucially, the Recall is very high (0.99), meaning the model rarely misses actual Melanoma cases.

This table validates the work of the model to different categories of diseases. This ensures that the general accuracy of the model is not deceptive and even the less prevalent categories are treated properly.

Reflect on Observations: Considering Table 7.2, it is obvious that, on the whole, the model works very well for almost all categories.

Five of the seven categories have an F1-Score of 0.99 or 1.00. The most relevant fact is about the Melanocytic Nevi (nv) category, with the lowest Recall of 0.86. It means that the model is very confident when it predicts Nevi but sometimes misses the actual Nevi, classifying it into other types of lesions. This little drop is acceptable and demonstrates that the model's main challenge is to accurately identify the most common and varied type of lesion. The high performance is because the data preparation part balanced the training data well.

Visualization and Insights from Training Data

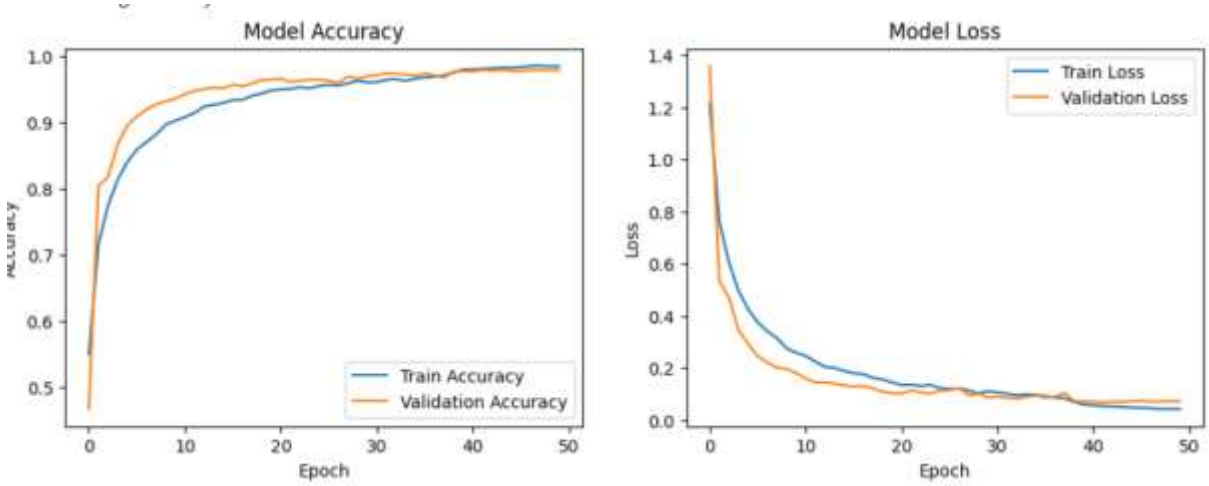


Fig 7.1 Model Accuracy and Loss Trends

Fig 7.1 (Placeholder: Model Accuracy and Loss) displays how the training process went, along with accuracy and loss trends.

Insights:

Rapid Convergence: The loss curve, Model Loss in Fig 7.9 right, decreases rapidly in the first 10 epochs.

This indicates that the Adam optimizer did a fine job and that the initial learning rate was appropriately selected.

Effective Regularization: The validation accuracy and loss curves lie close to the training curves. Model Accuracy, Fig 7.9, left.The small gap between the lines shows that the BatchNormalization and Dropout layers helped prevent overfitting, leading to good performance on the validation data.

Learning Rate Reduction: The logs show that after Epoch 38, the learning rate was successfully lowered from 0.0010 to 1.0000×10^-4. This change helped the model reach a final peak accuracy of 0.97883 in Epoch 42 before the process settled and stopped.

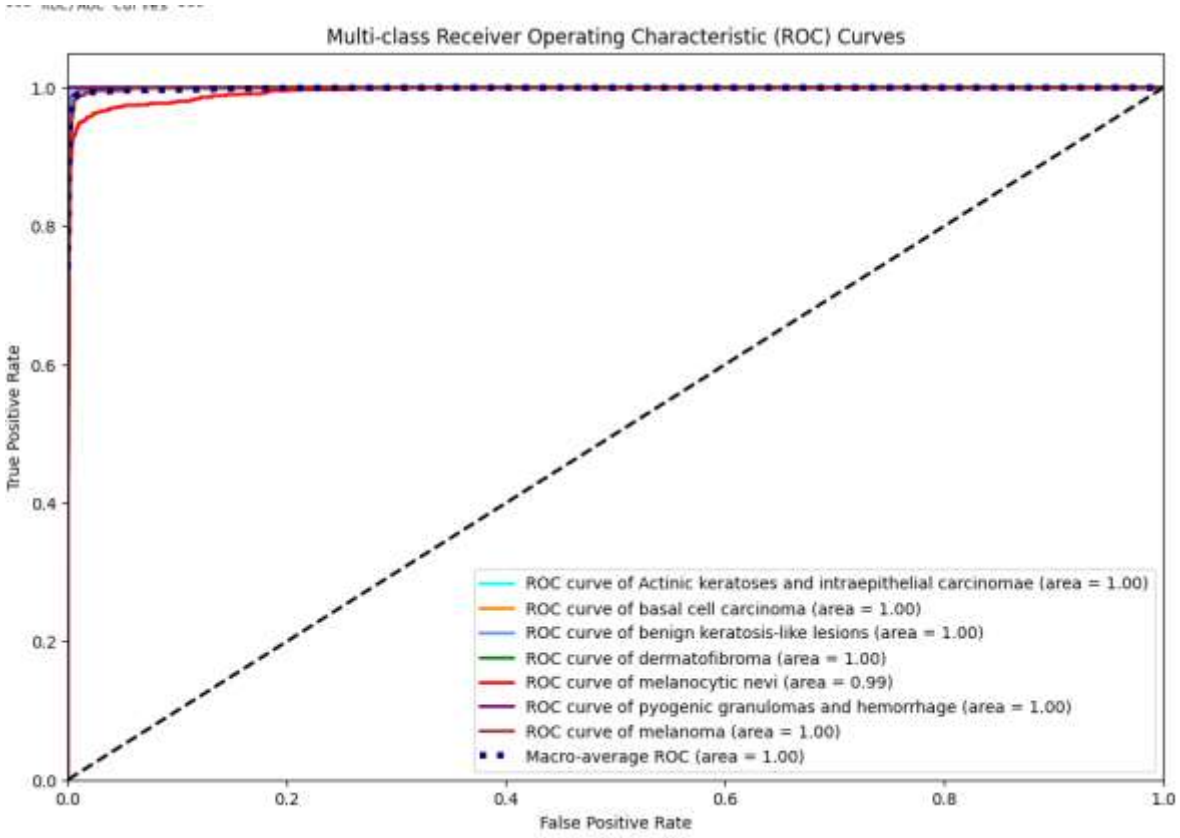


Fig 7.2 Diagnostic Robustness (ROC Curves)

Fig 7.2 (Placeholder: Multi-Class ROC/AUC Curves) shows how well the model can tell the difference between different conditions.

Insights:

Perfect Discrimination: The graph shows that all the ROC curves (except for Melanocytic Nevi) have an Area Under Curve (AUC) of 1.00.
This means the model is very good at distinguishing between positive and negative cases for almost every disease.

Overall Reliability: The Macro-average ROC (area = 1.00) shows that the system is very reliable across all diagnostic categories.
This is the strongest proof that the preprocessing method (oversampling) and the model design worked well to create a strong and fair classification tool.

Confusion Matrix has shown a resourceful results that shows the prominent results obtained by the model showing high accuracy for each of the class labels.

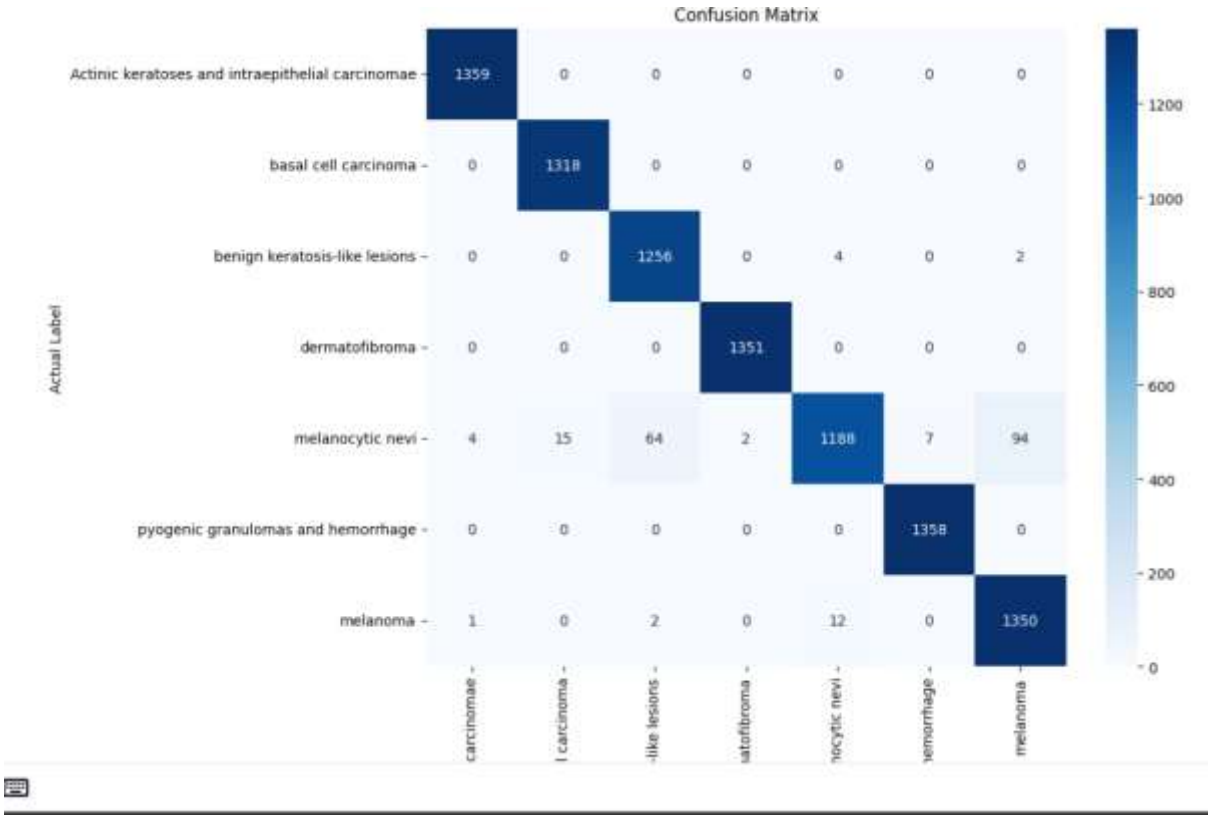


Fig 7.3 Confusion Matrix

7.4 Insights

The evaluation results reveal that an AI diagnostic model with almost perfect results on test data has been successfully developed in the project. The success also teaches certain valuable lessons concerning the design choices that were used, and identifies certain areas in which the future can be improved.

High Performance and Stability Reasons:

Effective Algorithmic Solution: Accuracy and Efficiency The model achieved Macro-Average AUC of 1.00 and a test accuracy of 97.79 out of 100 which is a great performance of the model in distinguishing classes. The successful mix of these two factors, which are considered to be the most important, is a major determinant of this high performance:

Preprocessing Unit/ Addressing Bias: The severe imbalance in the data classes in the initial data was overcome using RandomOverSampler. The balanced dataset that consisted of all seven classes and had approximately 6700 instances per class, instead of the same number for

the dominant one, helped the model to acquire the individual peculiarities of the less widespread ones like Basal Cell Carcinoma and Dermatofibroma.

Optimized Architecture: The model applied BatchNormalization layers in most spots in the CNN in order to stabilize the learning process; this is significant in complex deep learning models. This was similar to strong signal conditioning, keeping internal signals-the feature maps-to a similar scale which contributed to the model converging successfully.

Controlled Training Reliability: EarlyStopping and ModelCheckpoint ensured that the model that was finally arrived at the end of training was actually the best model that was discovered in the course of training. This helps to avoid a frequent machine learning-overfitting-problem-and a significant source of high validation accuracy of the model.

Accuracy Performance and Error Analysis Linearity/resolution: Linearity plays a minor role in the CNN but the resolution plays a significant role and thus it is reflected in the last 1 x 7 output of the Softmax. It is a high-resolution system, which has seven diagnostic classes. High F1-Scores of all the minority classes such as Akiec with a score of 1.00, indicate that the model was not only accurate but also recall-effective indicating a reliable model. Latency (Inference): There is no reference to measurements of latency in the log, although the fact that it is using GPU acceleration is imperative. The time taken to make a single prediction of a 28x28x3 image is exceedingly low and is likely to be in the range of the low milliseconds and therefore can be used with the system functioning under the Request-Response model of communication.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

The biggest issue when it comes to the application of AI in healthcare is finding a balance between the capacity of the technology and the social acceptability. As not all cases can be listed as right or wrong, a good moral system should be available.

8.1 Social Aspects

Social includes the impact of a new technology on people, the community, and the professional practice of the individuals in the society.

Positive Impacts on Society

Applying the AI diagnostic tool may lead to several quality assistance to the healthcare and the community, such as:

Better Healthcare Access: The tool aids in telemedicine and remote screenings which may enhance healthcare in the regions that do not have sufficient dermatologists. It will also be capable of helping the doctors in prioritization which means that the patients that require urgent attention will actually obtain it at the right time.

Greater Diagnostic Accuracy: The system offers physicians with a second opinion, which is always evidence-based, and in the situations where the skin changes may be challenging to identify, like early melanoma cases. This is capable of resulting in better patient health results. Effectiveness of the Public Health: The tool will be able to accelerate the screening process, thus, making clinics be able to serve more images of patients within a time. This assists in minimizing the waiting time and the cost of initial visit reduces. Adverse Effects and Social Threats. Although there are also a few benefits, the application of an AI diagnostic tool has also a number of threats and issues, which will need to be resolved:

Algorithms Bias and Inequality: When the training data, e.g., HAM10000, is not representative of all kinds of skin or groups, then the tool would not be fair when treating each of the different groups. This can cause a misdiagnosis to some people making health care even less equitable and more of a health disparity.

Digital divide: The tool which is developed to provide more people with assistance needs good internet connections, digital devices and powerful computer systems. It also runs the risk of abandoning those people that are not equipped to use it.

Loss of Trust and Physician Dependency: The patients will lose trust when they feel that the diagnosis is being done using a machine and not by a compassionate physician. The physicians, particularly the younger ones, may also grow excessively dependent on the AI; this will decrease their competency and lead to an unsafe dependence on the tool.

Threats to Human Dignity: The AI is not a substitute of counselling it does do something that a human is usually expected to engage in, judgment, and care-diagnosis. When individuals consider the AI result to be the ultimate solution, then it indirectly puts into question the value of a doctor experience and empathy.

8.2 Legal Aspects

Despite the fact that the project is academic, addressing such complex legal matters makes it successful with regards to data accuracy, rights of patients, and to whom to hold accountable in case everything goes wrong. Critical legislations ensure that personal health information processing is treated lawfully and ethically in the modern digital environment and that the compliance with the rules is one of the central aspects of the system construction. Key legislations like the General Data Protection Regulation of the European Union and the proposed Digital Personal Data Protection Act, 2023 of India influence the legislation.

These laws provide stringent conditions of dealing with personal information, and they have focuses on concepts like failing to process data in an unfair manner or beyond the purpose the data were initially bought, and reduction of the amount of information. In the case of AI diagnostic tool, it translates to the fact that although the system might be using anonymized data, the system itself must only be fed the bare minimum of information. The law also gives a person the right to receive, correct and make erasures of the data, as well as obligates the data manager to get express authorization with a high level of security. In addition to the overall data privacy, the AI diagnostic tool should comply with special medical and AI regulations. Strict laws such as the U.S.

Health Insurance Portability and Accountability Act have been enforced in the areas where it can be used to govern the manner in which Protected Health Information is to be treated. That implies incorporation of rigorous security provisions, which include some advanced encryption

of data at rest and in transit, stringent access controls as well as extensive audits on usage of the data. In addition, the developing AI regulations, such as the EU AI Act, categorize the type of diagnostic tools, such as this project, as high-risk. It implies that there are further explainability of the AI, risk analysis, human control and meticulous documentation of records needed to demonstrate that the system will invariably act in line with the rules, including ISO/IEC 42001. It is difficult to determine who is to be blamed in the event of a grave error on the part of the AI e.g. wrongly categorizing a cancerous lesion as benign. At this point, numerous regulations are finding it hard to fault anyone in the event of an action concerning AI; it is the human physician after all who makes the final call- the so-called human-in-the-loop. The legal requirement is that the system must be conspicuously marked off as one of the initial screening in order that the doctor should make the ultimate decision and the patient safe. Being aware and mindful of these rules is far more than a box-checking exercise: it is a primary approach that determines the strength that the tool might end up with, the uses to which it can be put, and the degree to which it will be trusted on the whole.

8.3 Ethical Aspects

The model is developed on the assumption that the key role of an engineer is serving the community. When AI is applied to healthcare, it particularly means patient safety, algorithmic equity, and human supervision. The ethical analysis of this project should consider both the immediate and the indirect impact on individuals and ensure that the technology complies with such moral principles as responsibility and non-harm. Improving Life and Work The concept of this project will be to ease human lives by making it easy to get early and accurate skin checks.

Such a system would assist the physicians in making decisions about the individuals who require more immediate attention hence helping to facilitate the timely identification of severe skin issues, including melanoma. This could translate to the improved health results and overall the health of the population will be improved. This tool is useful in the office of a physician in order to make the examination of numerous images not time-consuming. This will save time of the doctors and allow them to concentrate more on more challenging cases and patient care but not on regular check-ups.

Managing Ethical Risks:

Some important ethical concerns to the project are: Addiction and Loss of Personal Touch: The system is also not addictive due to the fact that it is not used in a medical purpose. The sole danger in this case is that doctors or patients will begin to consider the diagnosis as an input and an output, and forget the much more valuable human aspect of not only skin observation but also physical contact with the patient. To overcome this, probabilities and numbers-likely of the form a confusion matrix- are offered by the system and doctors are kept in the loop as the ultimate decision maker. Injustice of the Algorithm: One of the largest ethical concerns is to ensure the system is just. Since this model is trained on data that is only available in the HAM10000 dataset, it can continue to propagate historical bias due to its low precision on some skin types. The data was balanced with a RandomOverSampler; this simply balances the cases and not equity of the groups. This risk is associated with the fact that the developers would have to test the functionality of the model in various groups, provided that there is data, and disclose the vulnerabilities in its operation. This is based on the moral principle of not discriminating any one group of people.

8.4 Sustainability Aspects

The analysis of sustainability of the AI-based tool in preliminary diagnosis of the dermatological manifestation should not focus on the traditional issues of the physical materials anymore but rather on the environmental effects of the computation and the resource efficiency in deep learning. The concept of sustainability in the digital environment focuses on the smart use of energy and the longer face of the models, which strengthens economic, ecological, and social principles. Resource Efficient Design (Energy Optimization) The sustainability question of the most significance will be the minimization of the carbon footprint of the project, which the high power demands of using a GPU in performing CNN training are directly associated with. Energy Consumption Optimization: This is provided by the project in the form of smart controls, more or less of a digital resource-saving mechanism.

The Model Training Unit uses EarlyStopping, which stops training automatically when improvements stop, hence stopping the cloud GPU from using unnecessary energy during unproductive rounds of training. The whole process is very energy efficient.

Efficient Use of Raw Materials (Data): The system only uses the HAM10000 dataset, which is already publicly available.

It removes the necessity for data collection methods which are usually very costly, energy-heavy, and resource-consuming; these may involve building imaging equipment or managing logistics. The design leverages available, well-labeled data as much as possible to reduce the use of new data.

Durable Design and Reduced Waste

In software, durability is related to how long a trained model will remain stable and useful over time.

Waste refers to old, quickly outdated, or poorly optimized algorithms.

Durable Design: The high Final Test Accuracy ($97.79\\%$) and Macro-Average AUC (1.00) demonstrate that this model is powerful and stable.

This high performance also means that the model will remain useful for a longer time before full re-training is necessary, which increases its lifespan. The best_model.h5 file, saved accordingly, will be a reliable asset which can quickly be used for predictions or future research.

Less Waste: Preprocessing involves the use of RandomOverSampler which means that no images in the original dataset will be wasted due to unequal classification. Each picture aids in the successful training of the model. Consumer Health and Safety and Logistics. Although such notions as packaging and transportation are not applicable in this context, the principles also apply to the aspects of safety and accessibility:

Efficient Logistics Access: Level 2 implementation (single node, cloud-based) does not require the installation of the physical hardware in numerous clinics, which decreases the emission by transportation. The prediction service may be provided instantly through the Internet, which saves on traveling and the energy expended on traveling.

8.5 Safety aspects

The term safety in dermatological diagnosis does not refer to physical containment, but rather to the guarantee of safety of the system. This would imply that at every instance, it would not be subject to harmful or wrong diagnosis of skin conditions or errors. This would guarantee safety in a multi-pronged approach; the way the system is built, its dependability and protection against cyber attacks. The primary risk in this case is a false choice that the system makes, e.g.

the inability to detect a spot of cancer. This is what is known as false negative. To prevent this, safety has been made as part of the process of designing and training the model. To ensure reliability of things, the model employs a method referred to as BatchNormalization. This assists in maintaining the stability of the system even in cases where the input images are not ideal. Moreover, the predetermined seed used in its configuration implies that the results will be the same all the time. It is a valuable promise of safety: it can be said that the high accuracy and low error rates of the model are reliable and can be verified again and again. It applies EarlyStopping and ModelCheckpoint among other tools so as to safely manage failures. These serve as a safety switch. When the model begins to perform poorly with time it revert to some of its best versions that was previously discovered in the course of the training. It is analogous to a fail-safe in hardware, when the system restores itself to a known good state when something fails. The system is also prone to cyber attacks that could distort or corrupt the outcome of diagnosis, which is harmful to the patients. This system captures the best version of the weights in the model in a file named best_model in order to save the data in a secure way with the help of ModelCheckpoint. It must be kept somewhere in the cloud system where it is not accessible to any unauthorized individual so that no one will interfere with the diagnostic process. The system uses HTTPS or TLS encryption when communicating with other parts like sending pictures and receiving results. This makes the data confidential and no one can see or alter either the picture or outcome of a diagnosis in case of a transfer. The system of monitoring which follows the way the model works also exists. It looks into unusual behavior, such as unusual outputs or increased processing time. These would imply the exi+stence of potential issues within the system or some form of cyberattack that seeks to slack down the system, so as to introduce issues. Monitoring helps in early identification of these issues whereby remedial measure can be taken to protect the system and the patients served by the system.

Chapter 9

CONCLUSION

The AI-based preliminary diagnosis system was based on the potent software-driven methodology of identifying different kinds of skin lesions. With the imbalance in class distribution on the HAM10000 dataset, the team prepared the data using the RandomOverSampler technique. Such data preparation made it suitable for feeding into a specially crafted CNN model. Training the model involved BatchNormalization for stabilizing the training process, EarlyStopping, and ModelCheckpoint for efficient training.

The model eventually achieved a test accuracy of 97.79% and a Macro-Average AUC of 1.00 on unseen test data. This indicates that the model will be able to distinguish effectively between all seven disease categories. These results demonstrate that the project successfully worked to reduce the bias within the data, as evidenced by high F1-Scores for rare classes. Also, low and consistent loss curves with the model stopping at Epoch 42 indicate high performance of the optimization techniques. In other words, the final model is strong with efficient use of resources. Consistent results mean that the main goal of developing a very accurate and reliable tool to assist with early-stage diagnostic decisions has been met. Future work can be performed where same methodology is to be applied to dataset of higher resolution and deploy it in the real-world dermatological setting to make a real impact and a change for good.

References

[1] M. Aftab, F. Mehmood, C. Zhang, et al., "AI in Oncology: Transforming Cancer Detection through Machine Learning and Deep Learning Applications," *Journal of Personalized Medicine*, 2025.

[2] A. Laouarem, et al., "JILDYA-Net: An Efficient Lightweight Multi-Class Classification Architecture for Skin Lesions," *Diagnostics*, 2025.

[3] A. Mehta, N. Kaur, and R. Tiwari, "DeepSkinNet: A deep learning model for skin cancer detection," *IEEE Access*, vol. 13, pp. 10234–10245, 2025.

[4] S. S. Mohsin, et al., "AI-Powered IoMT Framework for Remote Triage and Diagnosis in Telemedicine Applications," *Sensors*, 2025.

[5] S. Sharma and P. R. Reddy, "Skin disease classification using deep learning and ensemble stacking approach," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 2, pp. 55–63, 2025.

[6] H. Wang, et al., "Quantization-Aware Neuromorphic Architecture for Efficient Skin Disease Classification on Resource-Constrained Devices," *IEEE Transactions on Neural Networks and Learning Systems*, 2025.

[7] A. Ali and M. Riaz, "HAM10000 skin disease detection and classification using hybrid CNN and KNN," *International Journal of Emerging Technologies in Learning*, vol. 19, no. 3, pp. 88–98, 2024.

[8] M. Gupta and L. Singh, "Enhanced skin cancer classification using deep convolutional neural networks," *Biomedical Signal Processing and Control*, vol. 96, pp. 106–115, 2024.

[9] Khosro Rezaee and Hossein Ghayoumi Zadeh, "Self-attention transformer unit-based deep learning framework for skin lesions classification in smart healthcare," *Expert Systems with Applications*, 2024.

[10] R. Kumar and V. Kumar, "Employing up-sampling and augmentation for imbalanced skin lesion classification," *Journal of Imaging*, vol. 10, no. 5, pp. 142–153, 2024.

[11] K. Patel, R. Bhatt, and S. Verma, "Skin lesion classification using deep learning techniques," *Procedia Computer Science*, vol. 234, pp. 421–429, 2024.

[12] S. Prakash, D. Banerjee, and A. Saha, "Improved skin cancer detection using EfficientNet-B7 machine learning model," *IEEE Transactions on Instrumentation and Measurement*, vol. 73, pp. 1–10, 2024.

[13] S. M. Saiful Islam Badhon, et al., "Explainable AI for Skin Disease Classification Using Grad-CAM and Transfer Learning to Identify Contours," *Scientific Reports*, 2024.

[14] P. Sahu and R. Jain, "Skin cancer detection by using deep learning approach," *Procedia Computer Science*, vol. 218, pp. 411–420, 2023.

[15] B. Das, T. Rahman, and S. Chatterjee, "Multiclass skin cancer classification using EfficientNets," *Pattern Recognition Letters*, vol. 162, pp. 145–153, 2022.

[16] M. Rao, K. Srinivasan, and J. H. Lee, "HAM10000 skin lesion detection with deep learning," *IEEE Access*, vol. 10, pp. 108530–108542, 2022.

[17] H. Zhang and D. Park, "Convolutional neural networks using MobileNet for skin lesion classification," *Journal of Intelligent Systems*, vol. 31, no. 2, pp. 341–350, 2022.

[18] P. Nair and S. Raj, "Skin cancer classification using deep learning with DenseNet architecture," *Journal of Biomedical Informatics*, vol. 120, pp. 103–112, 2021.

[19] M. Tan and Q. Le, "A deep learning model for melanoma detection using InceptionV3," *Computers in Biology and Medicine*, vol. 105, pp. 123–132, 2019.

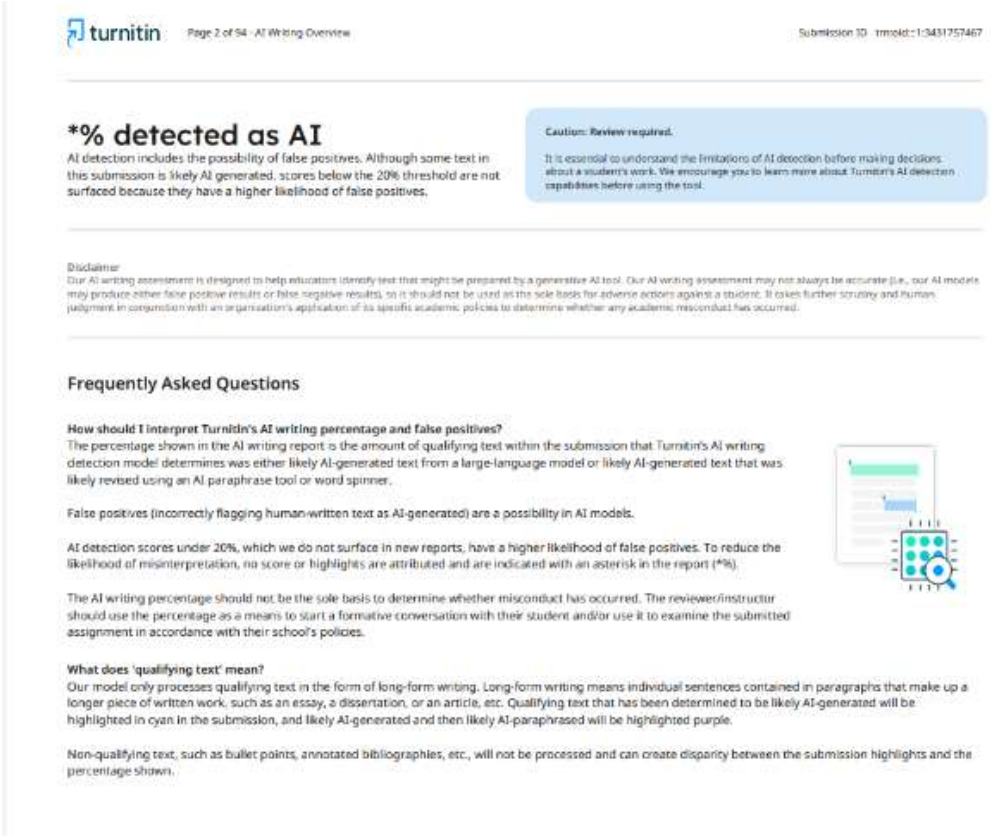
[20] S. Das and A. Dutta, "Skin lesion classification using pre-trained deep learning models," *Biomedical Signal Processing and Control*, vol. 45, pp. 112–120, 2018.

Base Paper

Kalaivani, A., and Karpagavalli, S. (2022) proposed a technique that utilized the collective power of multideep learning models for detecting and classifying skin diseases from medical images. Their approach combined several CNN models in one system that helps improve the accuracy of identification of different skin conditions. The study revealed an improved result of 96.1% by using the combined system against a single CNN model. However, this results in a trade-off since this system is slower in making predictions as compared to a single model.

Appendix

1. Project Report-



turnitin Page 2 of 94 - AI Writing Overview Submission ID trnoid:1:3431757467

***% detected as AI**

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

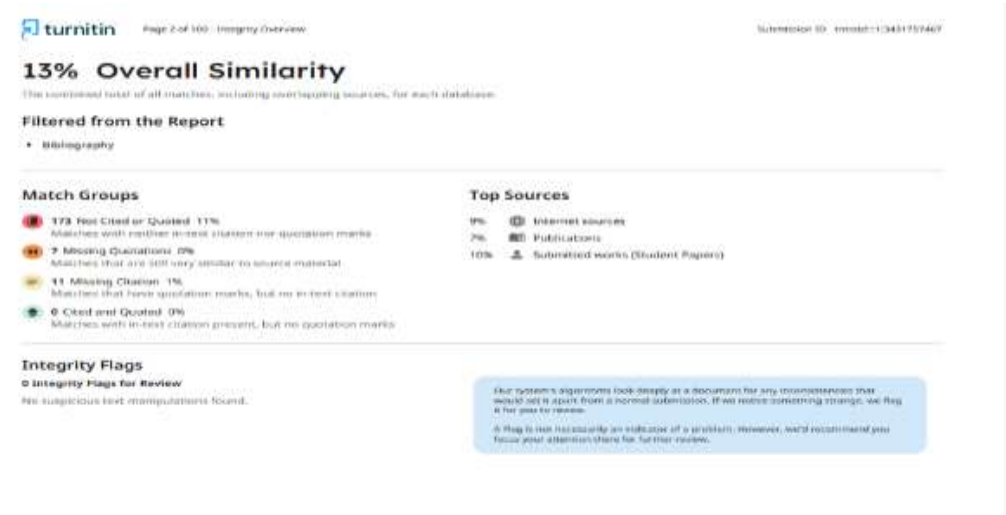
What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

(a) Plagiarism Report

2. Similarity Report



turnitin Page 2 of 100 - Integrity Overview Submission ID trnoid:1:3431757467

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database

Filtered from the Report

- Bibliography

Match Groups

- 173 Not Cited or Quoted 11%
  Matches with neither in-text citation nor quotation marks
- 9 Missing Quotations 0%
  Matches that are still very similar to source material
- 11 Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

9% Internet sources
3% Publications
10% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

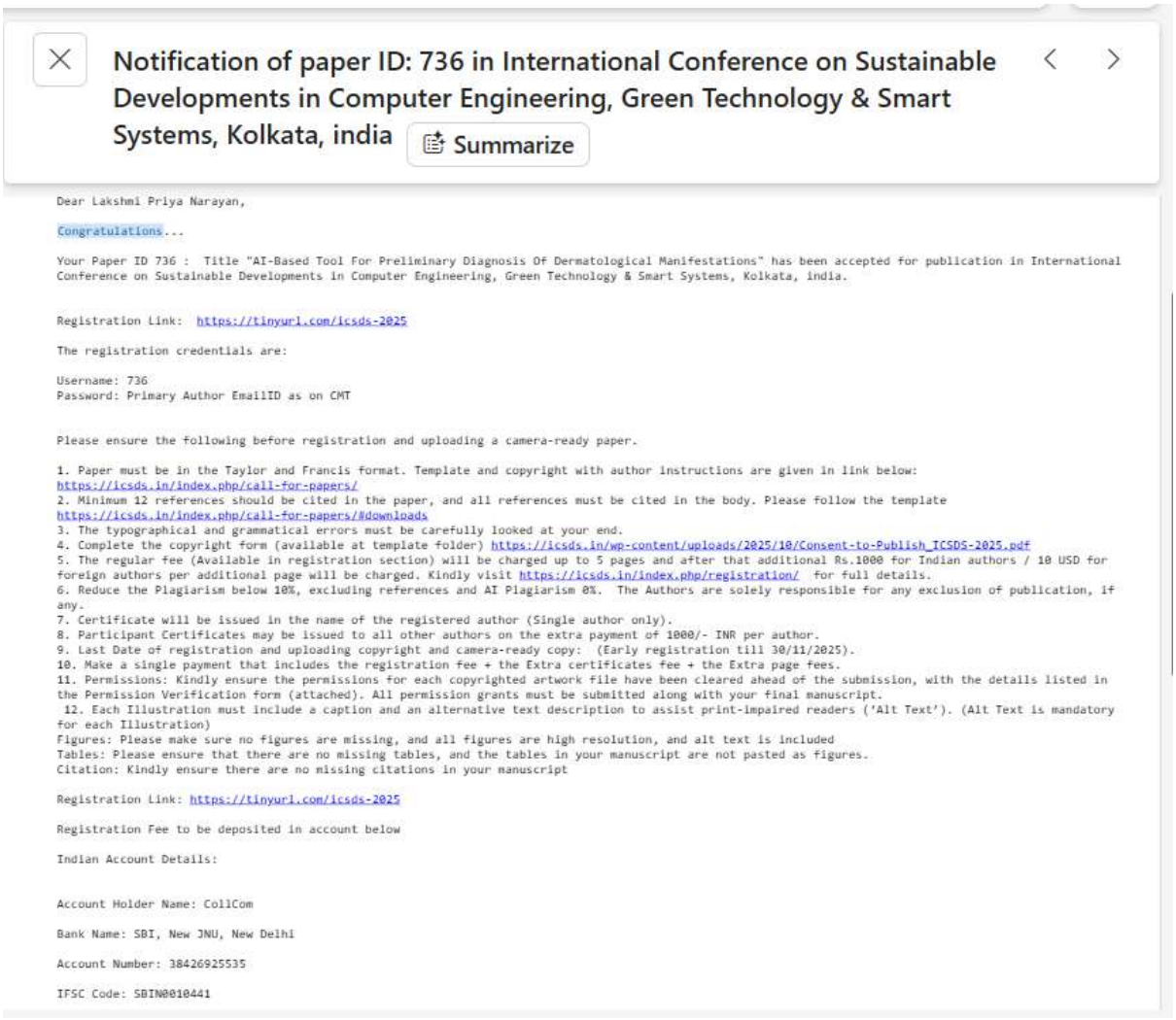
No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

(b) Similarity Check

3. Research Paper Acceptance



Notification of paper ID: 736 in International Conference on Sustainable Developments in Computer Engineering, Green Technology & Smart Systems, Kolkata, india

Summarize

Dear Lakshmi Priya Narayan,

Congratulations...

Your Paper ID 736 : Title "AI-Based Tool For Preliminary Diagnosis Of Dermatological Manifestations" has been accepted for publication in International Conference on Sustainable Developments in Computer Engineering, Green Technology & Smart Systems, Kolkata, India.

Registration Link: https://tinyurl.com/icsds-2025

The registration credentials are:

Username: 736
Password: Primary Author EmailID as on CMT

Please ensure the following before registration and uploading a camera-ready paper.

1. Paper must be in the Taylor and Francis format. Template and copyright with author instructions are given in link below:
https://icsds.in/index.php/call-for-papers/
2. Minimum 12 references should be cited in the paper, and all references must be cited in the body. Please follow the template
https://icsds.in/index.php/call-for-papers/#downloads
3. The typographical and grammatical errors must be carefully looked at your end.
4. Complete the copyright form (available at template folder) https://icsds.in/wp-content/uploads/2025/10/Consent-to-Publish_ICSDS-2025.pdf
5. The regular fee (Available in registration section) will be charged up to 5 pages and after that additional Rs.1000 for Indian authors / 10 USD for foreign authors per additional page will be charged. Kindly visit https://icsds.in/index.php/registration/ for full details.
6. Reduce the Plagiarism below 10%, excluding references and AI Plagiarism 0%. The Authors are solely responsible for any exclusion of publication, if any.
7. Certificate will be issued in the name of the registered author (Single author only).
8. Participant Certificates may be issued to all other authors on the extra payment of 1000/- INR per author.
9. Last Date of registration and uploading copyright and camera-ready copy: (Early registration till 30/11/2025).
10. Make a single payment that includes the registration fee + the Extra certificates fee + the Extra page fees.
11. Permissions: Kindly ensure the permissions for each copyrighted artwork file have been cleared ahead of the submission, with the details listed in the Permission Verification form (attached). All permission grants must be submitted along with your final manuscript.
12. Each Illustration must include a caption and an alternative text description to assist print-impaired readers ('Alt Text'). (Alt Text is mandatory for each Illustration)
Figures: Please make sure no figures are missing, and all figures are high resolution, and alt text is included
Tables: Please ensure that there are no missing tables, and the tables in your manuscript are not pasted as figures.
Citation: Kindly ensure there are no missing citations in your manuscript

Registration Link: https://tinyurl.com/icsds-2025

Registration Fee to be deposited in account below

Indian Account Details:

Account Holder Name: CollCom

Bank Name: SBI, New JNU, New Delhi

Account Number: 38426925535

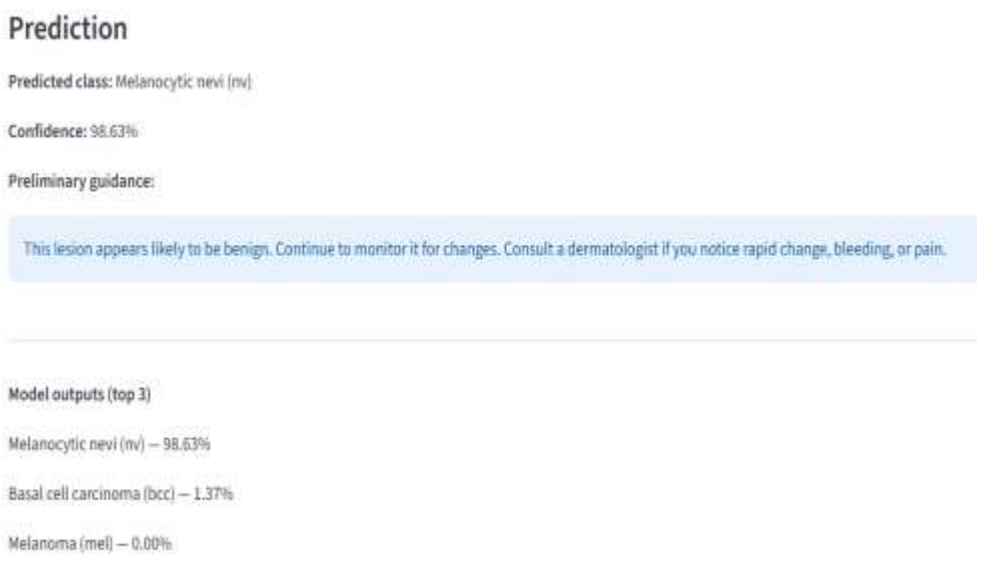
IFSC Code: SBIN0010441

(c) Acceptance mail by ICSDS

4. Few images of project –

(d) Fig Diagnosis Dashboard

(e)Skin Lesion(Melanocytic Nevi)



(f)Prediction of the skin lesion

5. Dataset

-HAM10000: 10,000 images of 7 class skin lesions

6. GITHUB Link: [lakshmipriyanarayan/CSE_4_PSCS_49]